

ADVERSARIAL ATTACKS LEVERAGE INTERFERENCE BETWEEN FEATURES IN SUPERPOSITION

Anonymous authors
Anonymous submission

ABSTRACT

Fundamental questions remain about when and why adversarial examples (AExs) arise in neural networks (NNs). Existing accounts typically frame vulnerability either as a defect of irregular decision landscapes or as sensitivity to non-robust input features. This paper contributes a third, mechanistic account: adversarial vulnerability can arise from the same *efficient representational compression* that lets networks encode many features in limited dimensionality. Our central claim is that superposition – representing more features than available dimensions – creates predictable interference patterns that adversaries can exploit. We make this contribution explicit in three steps. First, in synthetic models with controlled superposition, we show that superposition *suffices* to create adversarial vulnerability and that attack perturbations are predictable from the latent feature geometry. Second, we show that the same geometry explains two reviewer-relevant phenomena: transferability between models with similar training regimes and class-specific vulnerability patterns. Third, we test the mechanism in a vision transformer (ViT) trained on CIFAR-10 and find that bottleneck-induced superposition preserves the qualitative relationships between compression, robustness, transferability, and shared class geometry. Finally, by studying modular addition, we clarify the boundary of the claim: superposition is sufficient but not necessary, and representation-level mechanisms can also reveal algorithmic brittleness. Together, these results reframe some adversarial vulnerability as a byproduct of representational compression rather than solely as a learning flaw or a property of non-robust inputs. We release our implementation through a software repository.

1 INTRODUCTION

Despite extensive research on AExs (Szegedy et al., 2014; Goodfellow et al., 2014), their underlying cause remains unsettled. This gap is not only conceptual: without a mechanism, we cannot predict which small perturbations will succeed, why attacks transfer between independently trained models, or which design choices should lead to principled defences. The contribution of this paper is to make one such mechanism explicit and testable.

Existing explanations broadly fall into two camps (Nakkiran, 2019). The *bug* perspective attributes vulnerability to learning flaws (Schmidt et al., 2018; Nakkiran, 2019) or geometric properties of high-dimensional decision boundaries (Fawzi et al., 2016); it often yields statistical bounds on capacity (Bubeck & Sellke, 2021) or Lipschitz constraints (Hein & Andriushchenko, 2017), but does not identify which concrete perturbations should work. The *feature* perspective argues that AExs exploit predictive but non-robust statistical patterns in data (Ilyas et al., 2019), shifting attention to the relation between data and human perception. However, this view usually treats non-robust features as properties of the dataset rather than as consequences of how networks encode those properties. What remains missing is an account that connects representational constraints, data semantics, and attack geometry.

We fill this gap with a mechanistic interpretability account of adversarial vulnerability. Our central contribution is to show that AExs can arise from interference between learned representations in NNs. In particular, adversarial attacks can exploit superposed latent structure – a representational strategy that increases capacity by packing many features into few dimensions – and thereby manipulate outputs through predictable interference. The resulting pathway is reviewer-facing and testable: input correlations constrain feature arrangements; feature arrangements determine interference patterns; and interference patterns determine attack characteristics, robustness, and transferability.

This account builds on the linear representation hypothesis (LRH) (Park et al., 2024) and the theory of superposition (Elhage et al., 2022). The LRH proposes that *input features*—fundamental abstractions in the data—are represented as linear directions in activation space. Superposition explains how NNs can represent more such features than they have neurons by sharing dimensions, gaining representational efficiency while introducing interference. We ask whether this efficiency cost is also an adversarial vulnerability mechanism. Our contributions are:

- **Mechanism:** We present a framework linking architectural constraints and feature properties in adversarial vulnerability. In controlled models, data properties induce specific superposition geometries, and those geometries directly determine realised adversarial perturbations.
- **Predictions:** We show that interference patterns explain attack transferability and class-specific susceptibility: models with shared latent geometry share effective attacks, while features outside the relevant superposition geometry are not targeted.
- **Scale test:** We show that the qualitative mechanism persists in a ViT trained on CIFAR-10 with an engineered bottleneck layer that induces class-feature superposition.
- **Boundary case:** We show that superposition is sufficient but not necessary for adversarial vulnerability by identifying algorithmic brittleness in a modular-addition model with orthogonal features.
- **Actionability:** We demonstrate that mechanistic knowledge of learned representations can guide adversarial attack construction, motivating defences that target the relevant semantics rather than only norm-bounded perturbations.

2 BACKGROUND

This section introduces the representational assumptions used to state our contribution precisely. We use the LRH to connect semantic features to directions in activation space, superposition to describe compression-induced interference between those directions, and standard adversarial-robustness definitions to specify the attacks whose mechanisms we analyse. Formally, let $\mathbf{x} \in \mathcal{X}$ denote the input to a NN and $\mathbf{h}^{(l)} \in \mathbb{R}^{d_l}$ the activation vector of the l -th layer.

Linear representation hypothesis (LRH). The LRH provides the feature-level language needed for our mechanism: many variables used by a network, including semantic properties of inputs, are represented as linear directions in activation space and can therefore support geometric reasoning (Park et al., 2024; Guerner et al., 2023). This assumption is useful for our paper because adversarial perturbations can then be related to movements along, against, or between feature directions. The hypothesis is motivated by the computational convenience of linear separability and dot-product processing in later layers, and is supported by a growing body of evidence (Gurnee et al., 2023; Park et al., 2024). Let $\mathcal{C} = \{c_1, \dots, c_M\}$ denote a set of M semantically meaningful latent features (e.g. concepts like “presence of shape,” or “indoor vs. outdoor”). Formally:

Definition 1 (Linear Representation Hypothesis (LRH)) A neural network layer with activations $\mathbf{h}^{(l)} \in \mathbb{R}^{d_l}$ satisfies the LRH if it represents the latent features $\mathcal{C} = \{c_1, \dots, c_M\}$ as linear directions $\{\mathbf{v}_j\}_{j=1}^M \subset \mathbb{R}^{d_l}$ such that:

$$\mathbf{h}^{(l)}(\mathbf{x}) \approx \sum_{j=1}^M a_j(\mathbf{x}) \mathbf{v}_j$$

where $a_j(\mathbf{x}) \in \mathbb{R}$ represents the activation magnitude of feature c_j , and $\mathbf{v}_j$ is the corresponding linear direction. The features are **linearly accessible** (Costa et al., 2025): inputs $\mathbf{x}_0, \mathbf{x}_1 \in \mathcal{X}$ that differ mainly in the value of feature c_j while holding other features approximately constant (i.e., $a_j(\mathbf{x}_1) - a_j(\mathbf{x}_0) = k$ and $|a_i(\mathbf{x}_1) - a_i(\mathbf{x}_0)| < \lambda$ for all $i \neq j$ and some small $\lambda > 0$) satisfy:

$$\mathbf{h}^{(l)}(\mathbf{x}_1) - \mathbf{h}^{(l)}(\mathbf{x}_0) \approx k \mathbf{v}_j$$

where k reflects the change in c_j .

Superposition and sparsity. Superposition is the representational setting in which our main mechanism operates. NNs can encode more latent features than available activation dimensions, $M > d_l$; for instance, LLMs can refer to far more place names than they have residual-stream dimensions. In the superposition view, networks achieve this by using an overcomplete, non-orthogonal set of feature directions $\{\mathbf{v}_j\}_{j=1}^M$. This is possible because $2^{\Theta(d\epsilon^2)}$ almost orthogonal vectors ($< \epsilon$ cosine

similarity) can be represented in d -dimensional space (Tkocz, 2012), and sparse vectors can be accurately recovered after projection into lower-dimensional spaces (Elhage et al., 2022; Bereska & Gavves, 2024; Sawmya et al., 2025). The contribution-relevant cost is interference. First, *polyseman- ticity* (Scherlis et al., 2022; Lecomte et al., 2023) means that individual neurons can participate in multiple features, so a neuron is not itself a single interpretable concept. Second, non-orthogonal feature directions create *interference*: activating one feature partially activates others. Networks can reduce this ambiguity with nonlinear operations such as ReLU or softmax (Gurnee et al., 2023), but our experiments ask whether the remaining interference gives adversaries a predictable handle.

Definition 2 (Superposition Hypothesis) *A network layer represents features in superposition if:*

1. *The number of latent features exceeds layer dimensionality: $M > d_l$*
2. *There exists non-orthogonal feature directions: $\exists i, j$ with $i \neq j$ such that $\mathbf{v}_i \cdot \mathbf{v}_j \neq 0$*
3. *Latent feature activations are sparse: $\mathbb{E}_{\mathbf{x}}[\|\mathbf{a}(\mathbf{x})\|_0] \ll M$, where $\mathbf{a}(\mathbf{x}) = [a_1(\mathbf{x}), \dots, a_M(\mathbf{x})]^T$*

The network trades representational capacity against feature interference by packing more features than dimensions.

Adversarial attacks. We use standard adversarial-robustness terminology to connect the representa- tional mechanism to measurable failures (Szegedy et al., 2014; Costa et al., 2024):

Definition 3 (Adversarial attack) *Let $\mathbb{B}_p(\mathbf{x}, \epsilon)$ denote the ϵ -ball around $\mathbf{x} \in \mathcal{X}$ in the ℓ_p -sense, e.g. $\mathbb{B}_\infty$ is the ℓ_∞ -ball. Any small δ that p -perturbs $\mathbf{x}$ to create an **adversarial input** $\mathbf{x}' \in \mathbb{B}_p(\mathbf{x}, \epsilon)$ such that $f(\mathbf{x}') \neq f(\mathbf{x})$ (untargeted) or $f(\mathbf{x}') = y_{\text{target}}$ (targeted) is known to be an **adversarial attack**. ℓ_∞ - or ℓ_2 -norm attacks are created via the projected gradient descent (PGD) (detailed in Sec. A).*

Definition 4 (Adversarial robustness) *A classifier $g_w(\cdot)$ with parameters w is said to be **robust** at $\mathbf{x}$ with radius $\epsilon > 0$, if $g_w(\mathbf{x}') = g_w(\mathbf{x}) \forall \mathbf{x}' \in \mathbb{B}(\mathbf{x}, \epsilon)$.*

3 SUPERPOSITION GEOMETRY CAN DETERMINE ADVERSARIAL ATTACKS

This section establishes the paper’s first main contribution: in a controlled setting, superposition geometry is sufficient to produce predictable adversarial attacks. We use the toy model to make three claims testable:

1. *Do adversarial perturbations exploit the interference between superposed features?*
2. *Do correlations in the input shape the geometric arrangement of superposed latent features?*
3. *Can shared latent geometry explain why attacks transfer between independently trained models?*

Experimental setup. To isolate the mechanism, we design a synthetic task that reproduces established LRH and superposition phenomena (Park et al., 2024; Elhage et al., 2022) while keeping every relevant feature interpretable. This setup supports the contribution in four ways: (1) it gives an intuitive classification problem for AEs; (2) it represents class concepts as linear directions, as assumed by the LRH; (3) it induces controlled superposition between those directions; and (4) it preserves a priori knowledge of how inputs map to latent features, making adversarial mechanisms directly testable.

The task partitions an input $\mathbf{x} \in \mathbb{R}^d$ into k equally sized groups, $\mathbf{x} = [\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(k)}]$, where $\mathbf{x}^{(j)}$ contains the p input features associated with class j . The correct label is the group with the largest sum, $y = \operatorname{argmax}_{j \in \{1, \dots, k\}} \sum_{i=1}^p x_i^{(j)}$. Thus each $x_i^{(j)}$ is evidence for class j , and the p features represent different class attributes. Following Elhage et al. (2022), we sample $x_i^{(j)} \sim \text{Uniform}(0, 1)$ with sparsity S (probability of setting to zero).

The model then creates the intended representational pressure. A two-layer network compresses the input into m dimensions through a bottleneck encoder, $\mathbf{h} = \text{ReLU}(\mathbf{W}_e \mathbf{x} + \mathbf{b}_e) \in \mathbb{R}^m$ (where $m < k < d$), and reconstructs class logits with a decoder, $\mathbf{z} = \mathbf{W}_d \mathbf{h} + \mathbf{b}_d \in \mathbb{R}^k$. Our primary setup uses cross entropy (CE) loss without ReLUs or biases, because this simple case makes the interference mechanism easiest to analyse while the softmax exponentially suppresses interfering latent contributions. To check that the result is not an artefact of this choice, we also validate it with mean squared error (MSE) loss, ReLUs, and biases. In all cases, the core challenge is the same: compress $d = k \times p$ input dimensions into $m < k$ latent dimensions while retaining enough information to identify the group with the largest sum.

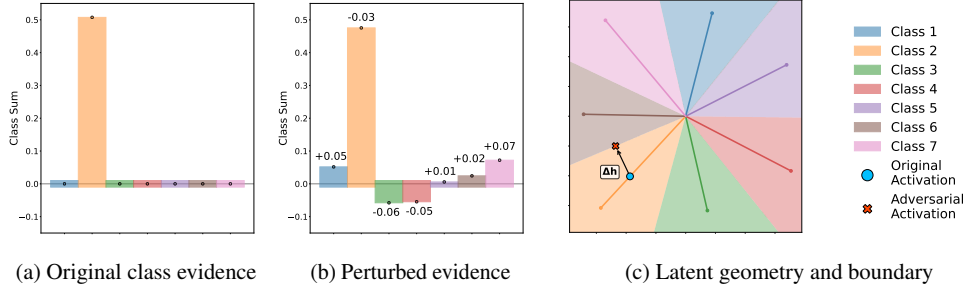


Figure 1: **Contribution mechanism: an attack exploiting superposition geometry** ($k = 7, m = 2$). (a) The original sample. (b) The adversarially perturbed sample: the ground truth is unchanged, but the model misclassifies it. The sign and magnitude of each input perturbation are determined by the latent representation geometry. (c) The same movement in activation space. Arrows denote the columns of $\mathbf{W}_e$, the latent representations of the input features.

This synthetic design also lets us separate true adversarial examples from perturbations that simply change the task label. In realistic data, latent features are nuanced and must be discovered; here, class features are labelled by construction. We can therefore recompute the true class after perturbation by recalculating group sums and define an AEx as satisfying two conditions:

1. The model’s prediction under perturbation changes (*e.g.* from class A to B)
2. The true class remains unchanged (*e.g.* still class A)

This criterion preserves the scientific target of the section: a successful attack must change the model’s prediction without changing which group truly has the largest sum. We generate attacks with ℓ_∞ - and ℓ_2 -norm PGD.

Evaluation metrics. Our metrics are chosen to make the proposed contribution inspectable rather than merely outcome-based. We interpret the columns of the encoder weight matrix $\mathbf{W}_e$ as latent representations $\mathbf{v}_i$ for input features x_i . Because input feature $x_i^{(j)}$ contributes only to class j , the corresponding $\mathbf{v}_i$ (for $i \in \text{class } j$) align with that class. This direct mapping lets us trace how perturbations to $\mathbf{x}^{(j)}$ move the latent representation. For successful PGD-generated AExs $\mathbf{x}_{adv} = \mathbf{x}_{orig} + \delta$, we measure:

1. **Input perturbation profile (IPP):** The sign and magnitude of each δ_i , describing how different input features are perturbed in an attack.
2. **Latent attack alignment:** The similarity $\Delta \mathbf{h} \cdot \mathbf{v}_c$ between the *latent attack vector* $\Delta \mathbf{h} = \mathbf{h}_{adv} - \mathbf{h}_{orig}$ and class c ’s representation.
3. **Attack transferability:** Success rate of attacks generated on one model when applied to another.
4. **Robust accuracy:** The fraction of examples that remain correctly classified after perturbation δ with $\|\delta\|_p \leq \epsilon$ to a sample of inputs.

3.1 RESULTS

Do attacks exploit interference between latent features in superposition? We first test the core mechanism: whether attack directions are determined by interference between superposed latent features. A decision-changing attack must move a sample across a boundary, but two simpler intuitions make different predictions. The *feature intuition* predicts that attacks should mainly increase target-class input features; the *bug intuition* predicts that perturbations should be idiosyncratic and not readable from feature representations.

Fig. 1 illustrates the reviewer-facing takeaway. An input from Class 2 (orange) is perturbed into an AEx classified as Class 6 (brown), and its IPP does not simply increase the target-class features. At the same time, the perturbation is not arbitrary: δ follows the configuration of the latent feature directions. The result supports a third interpretation, central to this paper: *attacks are mediated by latent geometry*.

The quantitative signature matches the visual mechanism. The IPP strongly correlates with latent attack alignment: the sign and magnitude of $\delta^{(j)}$ track how the class representation $\mathbf{v}^{(j)}$

Table 1: PGD-discovered attacks align with the theoretically optimal perturbations implied by superposition geometry. Results show mean $\pm$ std over 1000 attacks per condition. All p -values are below 10^{-10} .

k	m	Cosine Sim. (PGD vs Theory)	Cosine Sim. (Random Baseline)
6	2	0.97 ± 0.02	0.00 ± 0.02

aligns with the latent attack vector $\Delta \mathbf{h}$. Positively aligned representations ($\mathbf{v}^{(j)} \cdot \Delta \mathbf{h} > 0$) receive amplified input features, whereas negatively aligned representations are attenuated. Thus Fig. 1(b) is explained by the dot products in Fig. 1(c): the perturbation changes the input in exactly those directions that move the latent state across the boundary.

We then test the mechanism at scale within the toy family by comparing PGD-generated attacks to theoretically optimal perturbations derived from superposition geometry in Sec. 3.2. For each (k, m) configuration, we train 5 random seeds, generate 1000 PGD attacks per model, and compute cosine similarity between each successful perturbation and its corresponding optimum. As a baseline, we generate random perturbations with matching ℓ_2 norm and compare them to the same optimum. Tab. 1 shows near-perfect alignment between PGD and the optimal geometry-based attacks across dimensionalities; we filter out samples with ℓ_2 norm less than ϵ so that the comparison uses meaningful perturbations. One-sample t-tests against the random baseline give $p < 10^{-10}$ for every configuration. Because these optima exploit interference, the alignment shows that PGD is not merely finding arbitrary failures: it is recovering the perturbations predicted by superposition geometry.

Finding: Adversarial attacks systematically exploit interference between superposed features. Successful PGD attacks are predictable from the specific superposition geometry rather than random.

Do input correlations determine latent feature geometry? The second contribution of the toy section is to show where the attack-relevant geometry comes from. We systematically manipulate correlations in the training data to test how they constrain the degrees of freedom in learned representations. The *uncorrelated* condition samples features independently as above. The *paired* condition couples each feature in class i with a feature in class j , so that one activates when the other does. The *global* condition assigns each sample a random phase that induces a cyclic activation pattern: nearby class indices have similar activation probabilities and frequently co-activate, whereas distant classes rarely co-activate.

Fig. 2 shows that these correlations determine the arrangement of $\mathbf{v}^{(j)}$. We measure geometric similarity by comparing pairwise cosine-similarity matrices across feature pairs, then correlating those matrices across random seeds; higher values indicate more similar latent geometry. For each correlation condition, we test 25 seed pairs. In the $m = 2, k = 7$ setup, geometric similarity is 0.23 ± 0.14 (uncorrelated), 0.47 ± 0.32 (paired), and 0.88 ± 0.06 (global) (mean $\pm$ std). The monotonic pattern is the salient result: uncorrelated data leaves geometry highly variable, paired correlations partially constrain it, and global correlations force near-identical geometry. Pairwise two-sample t-tests with Bonferroni correction give $p < 10^{-3}$ for all comparisons ($\alpha = 0.05$ corrected for 3 comparisons). Results across additional (k, m) configurations are provided in Sec. C.3.

The mechanism is *interference avoidance*. Features that frequently co-activate are arranged to reduce mutual interference, so stronger correlations impose stronger geometric constraints. Uncorrelated data permits many class-separating arrangements; paired correlations fix relative positions of coupled features; and global correlations determine a unique geometry up to rotation.

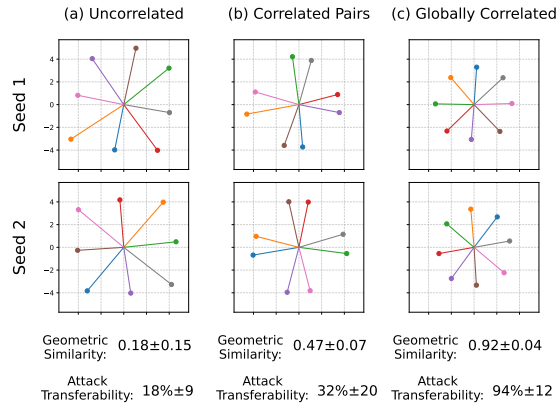


Figure 2: Input correlations make the contribution mechanism visible: stronger correlations create more consistent geometries between initialisations and increase attack transferability from 18% (uncorrelated) to 94% (global). Error bars show standard deviations.

Finding: Input correlations constrain feature geometry. Stronger correlations reduce geometric degrees of freedom, forcing different initialisations to converge to similar arrangements.

Does shared geometry explain attack transferability? The third toy-model contribution connects geometry to transferability. Having shown that input correlations induce consistent latent geometry across initialisations, we test whether the resulting shared interference patterns explain why attacks transfer. If attacks are tailored to interference patterns, they should transfer primarily between models with similar geometric arrangements.

Using the same seed pairs, we generate adversarial attacks on source models and evaluate them on target models within each correlation condition (25 transfer measurements per condition). All models achieve $> 95\%$ clean accuracy. Transfer rates track geometric similarity. For $k = 7, m = 2$ (Fig. 2), global correlations yield $94\% \pm 12\%$ attack transfer (mean $\pm$ std), uncorrelated data yields only $18\% \pm 9\%$, and paired correlations produce the intermediate value $32\% \pm 20\%$.

This result makes the transfer mechanism explicit. Each perturbation component amplifies or suppresses features so that, in the source model, their effects constructively interfere and cross the decision boundary. In a target model with different feature arrangements, the same components can cancel or interfere destructively, so the attack fails.

Finding: Attack transferability is governed by shared interference patterns.

Does reducing superposition suppress these attacks? If superposition creates vulnerability through interference, reducing superposition should reduce the relevant attacks. We test this claim with three interference-limiting experiments. First, we set $m = k$, allowing the network to learn orthogonal class directions $\mathbf{v}^{(j)}$. In this setting, any perturbation that changes the model’s prediction also changes the ground-truth class: moving a sample from class A to class B requires making the class-B feature sum exceed the class-A sum. Across 1000 attempts at every tested ϵ , we find zero successful adversarial examples. Second, we fix m and vary k to isolate superposition pressure from capacity; robust accuracy decreases monotonically with k/m (App. Tab. 5), showing that vulnerability scales with interference. Third, we constrain one class vector $\mathbf{v}^{(\perp)}$ to remain orthogonal to all others during training. When attacks are generated between classes that remain in superposition (e.g. from class a to b where $\mathbf{v}_a \not\perp \mathbf{v}_b$), the orthogonal class’s inputs remain unperturbed (Fig. 3).

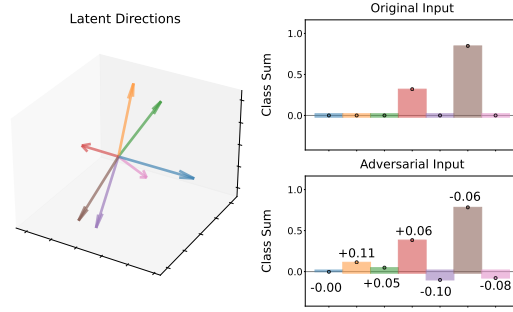


Figure 3: Class-specific contribution of the mechanism: an adversarial attack from class 5 to class 3 leaves the input features for an orthogonally represented class (class 1) unperturbed.

Finding: Adversarial attacks concentrate their budget on features that participate in the relevant superposition geometry.

3.2 FORMAL ANALYSIS

We next formalise the empirical contribution. In the linear setting, the relationship between superposition interference and adversarial vulnerability can be derived exactly, moving the argument from correlation to mechanism. We use the notation from the setup: $\mathbf{W}_e \in \mathbb{R}^{m \times d}$ induces superposition through columns $\mathbf{v}_i$, and the decoder corresponds to the encoder transposed, as observed empirically. The logit for class j is $z_j = \mathbf{v}_j^\top \mathbf{h}$.

Proposition 1. *The optimal input perturbations δ that maximise movement from class j to class k under constraint $\|\delta\|_2 = \epsilon$ satisfy $\delta \propto \mathbf{W}_e^\top \mathbf{n}$, where $\mathbf{n} = (\mathbf{v}_k - \mathbf{v}_j)$ is the normal to the decision boundary between classes.*

Proof Sketch. To move from class j to k , we maximise the logit margin $z_k - z_j = \mathbf{n}^\top (\mathbf{h} + \Delta \mathbf{h})$, where $\mathbf{n} = (\mathbf{v}_k - \mathbf{v}_j)$ and $\Delta \mathbf{h} = \mathbf{W}_e \delta$. This becomes $\max_{\|\delta\|_2 = \epsilon} \delta^\top \mathbf{W}_e^\top \mathbf{n}$. By Cauchy-Schwarz, this is maximised when $\delta \propto \mathbf{W}_e^\top \mathbf{n}$. Full proof in Sec. B. $\square$

Corollary 1 (Interference Drives Vulnerability) *The adversarial perturbation magnitude for feature i is $|\delta_i| \propto |\mathbf{v}_i^\top (\mathbf{v}_k - \mathbf{v}_j)|$, directly proportional to the differential interference between feature i and the class representations.*

This result states the mechanism in contribution form. Each input feature i is perturbed in proportion to its differential interference with the competing class representations. Because superposition makes feature directions non-orthogonal, even semantically unrelated features can affect the class decision; adversarial perturbations exploit exactly these dependencies to manipulate the output.

Proposition 2. *Models with feature representations related by orthogonal transformation $\mathbf{Q}$ (where $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$) share identical optimal attack directions in input space.*

Proof Sketch. Under transformation $\mathbf{v}'_i = \mathbf{Q}\mathbf{v}_i$, optimal perturbations remain invariant: $\delta'_i \propto (\mathbf{v}'_i)^\top (\mathbf{v}'_k - \mathbf{v}'_j) = \mathbf{v}_i^\top \mathbf{Q}^\top \mathbf{Q} (\mathbf{v}_k - \mathbf{v}_j) = \mathbf{v}_i^\top (\mathbf{v}_k - \mathbf{v}_j) = \delta_i$. Full proof in Sec. B. $\square$

Together with the experiments, these propositions state the section’s contribution as a **mechanistic pathway**: (i) input correlations constrain feature arrangements in superposition, (ii) those arrangements determine interference patterns, (iii) interference patterns dictate optimal perturbations via $\delta \propto \mathbf{W}_e^\top (\mathbf{v}_k - \mathbf{v}_j)$ (Proposition 1), and (iv) shared geometric constraints yield similar interference patterns across models, enabling transferability (Proposition 2):

Correlations $\xrightarrow{\text{constrain}}$ **Feature Geometry** $\xrightarrow{\text{determines}}$ **Interference Patterns** $\xrightarrow{\text{enable}}$ **Transferability**

4 ATTACKS IN REALISTIC MODELS

This section tests whether the toy-model contribution survives in a more realistic vision setting. We study a ViT (Dosovitskiy et al., 2020) trained on CIFAR-10 (Krizhevsky, 2009) (with ResNet-18 (He et al., 2016) results in Sec. D.2.1). By inserting a bottleneck layer immediately before the classifier, we force latent class features into superposition while leaving the rest of the model intact. This experiment asks whether the same contribution-relevant signatures appear outside the synthetic setting: (i) stronger superposition should reduce robustness and increase transferability, (ii) independently trained models should develop similar class-representation geometry, and (iii) attacks should affect features in a way consistent with that shared geometry.

Setup. We train the ViT architecture (6 layers, patch size 4, residual stream $d = 512$) on CIFAR-10 across five random seeds. After training, we replace the original classification head with a bottleneck: a linear encoder compresses pre-classification activations to an m -dimensional space, and a linear decoder expands them back to the $k = 10$ CIFAR-10 logits. As in Sec. 3, this bottleneck is trained with CE loss and no ReLU, forcing the k class representations into m -dimensional superposition while keeping the base ViT weights frozen. We vary $m \in \{2, 3, 5, 10\}$ to control superpositional compression. For each bottlenecked model, we generate AExs using both ℓ_∞ - and ℓ_2 -norm PGD, with a small step size, many iterations, and early stopping once the model prediction changes (details in Sec. D.1). To compare bottlenecks fairly, robust accuracy is normalised by clean accuracy; attack transferability follows the definition in Sec. 3.

Results. The base models reach 81% mean clean accuracy. As expected, bottleneck compression lowers clean accuracy to 54%, 67%, 76%, and 81% for $m = \{2, 3, 5, 10\}$, respectively.

The main result is that all three toy-model signatures reappear in the ViT setting. First, Fig. 4 (right) shows that smaller bottlenecks—that is, stronger superposition pressure—decrease normalised robust accuracy and increase attack transferability across random initialisations. The robustness drop is consistent with denser class packing: small input perturbations can induce broader interference among compressed class representations. The transferability increase matches the mechanism from Sec. 3: when fewer dimensions are available, there are fewer viable geometric arrangements for class features, so independently trained models converge to more similar interference patterns. Results across perturbation magnitudes are reported in Sec. D.1.1.

Second, Fig. 4 (left) shows that the m -dimensional class representations in the bottleneck have consistent ordering and clustering across models. As in the toy experiments in Sec. 3, this consistency suggests that data correlations constrain the superposed geometry; the clusters often also reflect semantic class similarity, such as ‘cat’ and ‘dog’ representations appearing together. Third, AExs

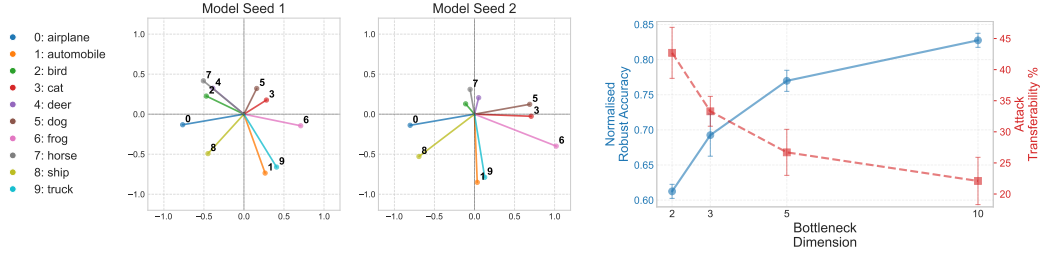


Figure 4: ViT bottlenecks reproduce the paper’s contribution pattern. Left: CIFAR-10 class representation structure remains similar across model seeds. Right: increasing bottleneck dimension reduces attack transferability and increases robust accuracy.

generated for a bottlenecked model produce similar relative logit changes when passed through the corresponding base ViT, measured as $\text{logit}_{\text{base}}(\mathbf{x}_{\text{adv}}) - \text{logit}_{\text{base}}(\mathbf{x}_{\text{orig}})$. This indicates that the perturbations are not merely artefacts of the added bottleneck: the base model responds to them in a consistent direction, even though without the extra compression they do not usually cause misclassification.

5 ADVERSARIAL VULNERABILITY VIA ALGORITHMIC BRITTLINESS

The preceding sections establish that feature superposition can *suffice* for adversarial vulnerability. This section clarifies the boundary of that contribution by asking whether superposition is *necessary*. We study a setting where the relevant features are represented orthogonally rather than in superposition, yet the model remains vulnerable. The contribution of this section is therefore twofold: it separates superposition-specific vulnerability from a broader representation-level analysis of attacks, and it shows that mechanistic understanding can produce gradient-free attack constructions. We ask: *how do such attacks work mechanistically, can the mechanism be used to construct attacks directly, and does naive robustness certification remove the failure mode or only raise the required attack budget?*

Setup. We train a linear encoder + MLP on one-hot integers (a, b) to predict $(a + b) \bmod P$. As detailed in Sec. E and Nanda et al. (2023), the model learns a trigonometric algorithm for the task. As part of this learned algorithm¹ the encoder projects inputs onto non-basis-aligned linear representations encoding $\sin(\omega_k a)$ and $\cos(\omega_k a)$ at key frequencies $\omega_k = 2\pi k/P$. The MLP then composes $\cos((a + b)\omega_k)$ through sum-angle identities and reaches 100% accuracy. Crucially for this boundary case, these features are represented *orthogonally* (i.e. not in superposition). We generate AExs by relaxing the one-hot constraint and perturbing only input a with PGD, thereby moving inputs off the training manifold.

Attacks exploit the learned key frequencies. Fourier analysis of successful perturbations δ (App. Fig. 6) shows that their dominant frequencies match the learned algorithm’s key frequencies, ω_k . Attacks do not transfer to models with different ω_k (for example, models trained from different seeds), yielding 0% transfer. The mechanism is therefore algorithm-specific: frequency-aligned perturbations effectively alter the sinusoidal values read from the encoder features. The perturbation δ turns one-hot $\mathbf{x}_a$ into a dense vector $\mathbf{x}'_a$, whose newly non-zero entries corrupt the sinusoidal values consumed by the MLP and cause misclassification. Because the model’s logits use constructive interference across each ω_k , disrupting only one ω_k is insufficient; successful attacks must alter multiple frequency components together. Sec. E.1 gives the detailed mechanistic intuition.

Analytic construction of attacks. The frequency mechanism is actionable. We construct gradient-free, *informed* perturbations as composite sinusoidal waves using the matching ω_k . On $0 + 0 \bmod P$, these attacks succeed with $\|\delta\|_2 = 0.22$, close to PGD (0.17, $\approx 1.3 \times$ the PGD budget) and far below naive alternatives (single-index 0.97, ℓ_∞ noise 2.04, uniform 6.20). This supports the claim that the identified mechanism is the relevant one and shows that representation-aware analysis can yield attack recipes, not only post-hoc explanations.

¹We refer to the specific computation executed by the network to solve the task as the *model’s learned algorithm*. As this algorithm depends on extracting precise values from learned feature representations (frequencies), it remains *brittle* in the face of small perturbations to the bases. We term this the *algorithmic brittleness*.

Do defences mitigate this algorithmic vulnerability? We next test whether certified training changes the mechanism. We apply certified training against ℓ_∞ perturbations (Müller et al., 2023). For fixed attack budgets, *robust* accuracy improves after robustification: $0\% \rightarrow 85\%$ at $\epsilon_\infty = 5e-3$, $0\% \rightarrow 96\%$ at $2e-3$, and $76\% \rightarrow 100\%$ at $1e-3$. However, PGD still attacks the robust model through the same frequency-based brittleness, now requiring larger perturbation norms. Certification therefore raises the budget needed to exploit the mechanism but does not remove the mechanism itself, motivating defences that target identified semantics—here, frequency manipulation—rather than relying only on broad norm bounds.

Relation to the paper’s thesis. This section makes the contribution boundary explicit. First, superposition is *sufficient but not necessary*: even orthogonal representations can be adversarially brittle. Second, the broader architecture–semantics perspective is useful beyond superposition, because it guides both attack design through representation-informed perturbations and defence design through mechanism-specific targets.

Although superposition is less central in this example, sparsity still matters: upstream violations of sparsity (*e.g.* noise or superposed features) can deliver effectively ‘pre-perturbed’ inputs to downstream orthogonal circuits and activate the same frequency-based failure.

6 RELATED WORK

Superposition & latent representations. Our contribution builds on work showing that feature representations can share dimensions and interfere. Nanda (2023) distinguish *representational* from *computational* superposition, while Gurnee et al. (2023) distinguish *alternating* from *simultaneous* interference. Prior work also shows that data correlations shape representational arrangements: correlated feature pairs become orthogonal (Elhage et al., 2022), correlations can drive the formation of superposition (Chan, 2024), and semantic clustering can emerge from correlation structure (Prieto et al., 2025a). More broadly, representation geometry is influenced by spectral bias (Rahaman et al., 2019), neural collapse (Kothapalli, 2023), and optimisation objectives (Casper, 2023). Sparse autoencoders (SAEs) provide one tool for disentangling superposed features in vision models (Lim et al., 2024; Joseph et al., 2025). We differ from this literature by using superposition geometry to predict adversarial perturbations, transferability, and class-specific vulnerability.

Adversarial vulnerability. The adversarial-robustness literature is often organised around attacks and defences. Attacks (Szegedy et al., 2014) use imperceptible perturbations to maximise model error, with standard methods including FGSM (Goodfellow et al., 2014) and PGD (Madry et al., 2018). Robustness work asks models to maintain performance under such perturbations, using defences such as adversarial training (Madry et al., 2018) and certified training (Wong & Kolter, 2018; Palma et al., 2024), which provide guarantees within bounded input regions. Our contribution is complementary: rather than proposing a new attack or defence benchmark, we identify a representational mechanism that can explain why particular attacks work.

The role of input & latent features on adversarial robustness. Several accounts attribute vulnerability to predictive but brittle ‘non-robust’ input features (Ilyas et al., 2019; Goh, 2019) or to shortcut learning (Li et al., 2023). Perturbation characteristics also vary by dataset (Maiya et al., 2021). Elhage et al. (2022) suggest that superposition may connect to AExs, while Casper (2023) debate the relationship between interpretability and adversarial examples. A tension between representational efficiency and robustness has also been observed (Barsbey et al., 2025). Closest to our framing, Zhang et al. (2021) show that adversarial perturbations move latent representations across decision boundaries, and Ganeshan et al. (2019) find that PGD targets final layers and propose feature-based attacks. Transferability is known to be limited by representation discrepancies between models (Li et al., 2023; Wang et al., 2024), and decorrelating features reduces transfer (Wiedeman & Wang, 2022). Our paper makes these connections mechanistic by tying transfer to shared interference patterns produced by constrained superposition geometry.

7 DISCUSSION & CONCLUDING REMARKS

This paper contributes a mechanistic account of adversarial vulnerability: attacks can exploit interference patterns created by the geometry of superposed features in NNs. Across controlled toy models and bottlenecked ViTs, we show that data properties such as correlations and sparsity induce distinct superposition geometries, and those geometries make adversarial perturbations, transferability, and

class-specific susceptibility more predictable. The modular-addition study sharpens the claim by showing that superposition is sufficient but not necessary; representation-level mechanisms can also expose algorithmic brittleness in orthogonal circuits. Taken together, the results reframe some adversarial vulnerability as a consequence of how networks efficiently encode information, rather than solely as a flaw in optimisation or a fixed property of non-robust input features.

Limitations & future work. The main limitation is scope. Our strongest evidence comes from simplified or engineered settings where the relevant features and bottlenecks are known, while large real networks contain unknown, unlabelled features interacting across many layers. The broader adversarial-robustness literature also suggests that vulnerability mechanisms may differ across model families and attack types. For example, constrained attacks such as universal perturbations, jailbreaks, or patch attacks may be unable to activate the specific feature combinations needed to exploit superposition interference.

Future work should quantify the ‘adversarial cost’ of superposition, connect that cost to accuracy–robustness trade-offs, and test how robust training reshapes superposition geometry. A key next step is to move from engineered class-feature superposition to naturally occurring, unlabelled features in large networks. We plan to use SAEs (Bricken et al., 2023) for this purpose; preliminary results in Sec. F suggest that this is a promising route for making the contribution mechanism observable at larger scale.

ETHICS STATEMENT

This work is primarily mechanistic and interpretive. Its contribution is to explain existing adversarial phenomena through representation geometry, not to introduce a new attack capability. The experiments use only public CIFAR-10 data and synthetic toy or modular-arithmetic tasks, with no human subjects or personally identifiable information. The gradient-based attacks used here are standard in the literature. By clarifying when vulnerability may arise from representational compression or algorithmic brittleness, we aim to support future work on robustness mechanisms and defences that balance model capability with trustworthy deployment. We have no conflicts of interest to declare.

REPRODUCIBILITY STATEMENT

We include source code for all experiments in the supplementary materials. The toy model experiments (Sec. 3) provide the architectural details, training procedures, evaluation metrics, and data-generation processes needed to reproduce the superposition-geometry results. The ViT experiments (Sec. 4) specify the architecture and bottleneck-training procedure, with additional details in Sec. D.1. The modular-arithmetic experiments (Sec. 5) describe the network architecture, data encoding, learned trigonometric algorithm, and attack construction, with further details in Sec. E.2. Experiments were conducted on an AMD Ryzen Threadripper 3970X 32-Core with 256GB RAM and RTX 3090. Additional ResNet-18 and SAE experiments are documented in Appendices Sec. D.2.1 and Sec. F, respectively.

LLM USAGE STATEMENT

LLMs assisted with prior literature search, LaTeX table generation from CSV files (with all numerical values verified), and phrasing suggestions for manuscript clarity. The scientific contributions, experimental design, analysis, and conclusions remain the original work of the authors.

REFERENCES

- Melih Barsbey, Antônio H Ribeiro, Umut Şimşekli, and Tolga Birdal. On the interaction of compressibility and adversarial robustness. *arXiv preprint arXiv:2507.17725*, 2025.
- Leonard Bereska and Stratis Gavves. Mechanistic interpretability for AI safety - a review. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856.
- Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermy, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023.
- Sébastien Bubeck and Mark Sellke. A universal law of robustness via isoperimetry. In Marc’Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan (eds.), *Advances in Neural Information Processing Systems*, pp. 28811–28822, 2021.
- Stephen Casper. EIS IX: Interpretability and adversaries. *AI Alignment Forum*, February 2023.
- Lawrence Chan. Superposition is not “just” neuron polysemanticity. *AI Alignment Forum*, April 2024.
- Joana C Costa, Tiago Roxo, Hugo Proença, and Pedro Ricardo Morais Inacio. How deep learning sees the world: A survey on adversarial attacks & defenses. *IEEE Access*, 12:61113–61136, 2024.
- Valérie Costa, Thomas Fel, Ekdeep Singh Lubana, Bahareh Tolooshams, and Demba Ba. From flat to hierarchical: Extracting sparse representations with matching pursuit. *arXiv preprint arXiv:2506.03093*, 2025.

594 Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hier-
595 archical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*,
596 pp. 248–255, 2009.

597 Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas
598 Unterthiner, Mostafa Dehghani, Matthias Minderer, G Heigold, S Gelly, et al. An image is
599 worth 16x16 words: Transformers for image recognition at scale. In *International Conference on*
600 *Learning Representations*, 2020.

601
602 Rüdiger Ehlers. Formal verification of piece-wise linear feed-forward neural networks. In Deepak
603 D’Souza and K. Narayan Kumar (eds.), *Automated Technology for Verification and Analysis -*
604 *15th International Symposium, ATVA 2017, Pune, India, October 3-6, 2017, Proceedings*, volume
605 10482 of *Lecture Notes in Computer Science*, pp. 269–286. Springer, 2017.

606 Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec,
607 Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, et al. Toy models of superposition.
608 *arXiv preprint arXiv:2209.10652*, 2022.

609
610 Alhussein Fawzi, Seyed-Mohsen Moosavi-Dezfooli, and Pascal Frossard. Robustness of classifiers:
611 from adversarial to random noise. *Advances in neural information processing systems*, 29, 2016.

612
613 Aditya Ganeshan, Vivek BS, and R Venkatesh Babu. FDA: Feature disruptive attack. In *Proceedings*
614 *of the IEEE/CVF International Conference on Computer Vision*, pp. 8069–8079, 2019.

615 Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever,
616 Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. In *The Thirteenth*
617 *International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*,
618 2025.

619
620 Gabriel Goh. A discussion of ‘adversarial examples are not bugs, they are features’: Two examples
621 of useful, non-robust features. *Distill*, 2019.

622
623 Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial
624 examples. *arXiv preprint arXiv:1412.6572*, 2014.

625
626 Clément Guerner, Anej Svete, Tianyu Liu, Alexander Warstadt, and Ryan Cotterell. A geometric
627 notion of causal probing. *arXiv preprint arXiv:2307.15054*, 2023.

628
629 Wes Gurnee, Neel Nanda, Matthew Pauly, Katherine Harvey, Dmitrii Troitskii, and Dimitris Bertsimas.
630 Finding neurons in a haystack: Case studies with sparse probing. *Trans. Mach. Learn. Res.*, 2023.

631
632 Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image
633 recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*,
634 pp. 770–778, 2016.

635
636 Matthias Hein and Maksym Andriushchenko. Formal guarantees on the robustness of a classi-
637 fier against adversarial manipulation. In *Advances in Neural Information Processing Systems*,
638 volume 30, 2017.

639
640 Patrick Henriksen and Alessio Lomuscio. Robust training of neural networks against bias field
641 perturbations. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(12):14865–14873,
642 Jun. 2023.

643
644 Nikolaus Howe, Ian McKenzie, Oskar Hollinsworth, Michał Zajac, Tom Tseng, Aaron Tucker,
645 Pierre-Luc Bacon, and Adam Gleave. Scaling trends in language model robustness, 2025.

646
647 A. Ilyas, S. Santurkar, D. Tsipras, L. Engstrom, B. Tran, and A. Madry. Adversarial examples are not
648 bugs, they are features. In *Advances in Neural Information Processing Systems 32*, pp. 125–136,
649 2019.

650
651 Sonia Joseph, Praneet Suresh, Lorenz Hufe, Edward Stevinson, Robert Graham, Yash Vadi, Danilo
652 Bzdok, Sebastian Lapuschkin, Lee Sharkey, and Blake Aaron Richards. Prisma: An open source
653 toolkit for mechanistic interpretability in vision and video, 2025.

- Guy Katz, Clark W. Barrett, David L. Dill, Kyle Julian, and Mykel J. Kochenderfer. Reluplex: An efficient SMT solver for verifying deep neural networks. In Rupak Majumdar and Viktor Kuncak (eds.), *Computer Aided Verification - 29th International Conference*, volume 10426, pp. 97–117. Springer, 2017.
- Vignesh Kothapalli. Neural collapse: A review on modelling principles and generalization. *Trans. Mach. Learn. Res.*, 2023.
- Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009. CIFAR-10 dataset.
- Victor Lecomte, Kushal Thaman, Rylan Schaeffer, Naomi Bashkansky, Trevor Chow, and Sanmi Koyejo. What causes polysemanticity? an alternative origin story of mixed selectivity from incidental causes. *arXiv preprint arXiv:2312.03096*, 2023.
- Ang Li, Yifei Wang, Yiwen Guo, and Yisen Wang. Adversarial examples are not real features. *Advances in Neural Information Processing Systems*, 2023.
- Hyesu Lim, Jinho Choi, Jaegul Choo, and Steffen Schneider. Sparse autoencoders reveal selective remapping of visual concepts during adaptation. *arXiv preprint arXiv:2412.05276*, 2024.
- Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Pearce, Nicholas L. Turner, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Hoagy Cunningham, Thomas Henighan, Adam Jermyn, Andy Jones, Andrew Persic, Zhenyi Qi, T. Ben Thompson, Sam Zimmerman, Kelley Rivoire, Thomas Conerly, Chris Olah, and Joshua Batson. On the biology of a large language model. *Transformer Circuits Thread*, 2025.
- Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards Deep Learning Models Resistant to Adversarial Attacks. In *International Conference on Learning Representations*, February 2018.
- Shishira R. Maiya, Max Ehrlich, Vatsal Agarwal, Ser-Nam Lim, Tom Goldstein, and Abhinav Shrivastava. A Frequency Perspective of Adversarial Robustness. *arXiv*, 2021.
- Mark Niklas Müller, Franziska Eckert, Marc Fischer, and Martin T. Vechev. Certified training: Small boxes are all you need. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.
- Preetum Nakkiran. A discussion of ‘adversarial examples are not bugs, they are features’: Adversarial examples are just bugs, too. *Distill*, 2019.
- Neel Nanda. A comprehensive mechanistic interpretability explainer and glossary, 2023.
- Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.
- Alessandro De Palma, Rudy Bunel, Krishnamurthy Dvijotham, M. Pawan Kumar, and Robert Stanforth. IBP regularization for verified adversarial robustness via branch-and-bound. *CoRR*, abs/2206.14772, 2022.
- Alessandro De Palma, Rudy Bunel, Krishnamurthy (Dj) Dvijotham, M. Pawan Kumar, Robert Stanforth, and Alessio Lomuscio. Expressive losses for verified robustness via convex combinations. In *The Twelfth International Conference on Learning Representations, ICLR, 2024*.
- Kiho Park, Yo Joong Choe, and Victor Veitch. The linear representation hypothesis and the geometry of large language models. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net, 2024.
- L Prieto, E Stevinson, M Barsbey, T Birdal, and P Mediano. Correlations in the data lead to semantically rich feature geometry under superposition. *Mechanistic Interpretability Workshop at NeurIPS 2025*, 2025a.

-
- Lucas Prieto, Melih Barsbey, Pedro A. M. Mediano, and Tolga Birdal. Grokking at the edge of numerical stability. In *The Thirteenth International Conference on Learning Representations, ICLR, 2025b*.
- Aditi Raghunathan, Jacob Steinhardt, and Percy Liang. Certified defenses against adversarial examples. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net, 2018.
- Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred A. Hamprecht, Yoshua Bengio, and Aaron C. Courville. On the spectral bias of neural networks. In *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, volume 97 of *Proceedings of Machine Learning Research*, pp. 5301–5310. PMLR, 2019.
- Shashata Sawmya, Linghao Kong, Ilia Markov, Dan Alistarh, and Nir N Shavit. Wasserstein distances, neuronal entanglement, and sparsity. In *The Thirteenth International Conference on Learning Representations, 2025*.
- Adam Scherlis, Kshitij Sachan, Adam S Jermyn, Joe Benton, and Buck Shlegeris. Polysemanticity and capacity in neural networks. *arXiv preprint arXiv:2210.01892*, 2022.
- Ludwig Schmidt, Shibani Santurkar, Dimitris Tsipras, Kunal Talwar, and Aleksander Madry. Adversarially robust generalization requires more data. In *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, pp. 5019–5031, 2018.
- C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus. Intriguing properties of neural networks. In *2nd International Conference on Learning Representations, ICLR 2014, 2014*.
- Tomasz Tkocz. An upper bound for spherical caps. *The American Mathematical Monthly*, 119(7): 606–607, 2012.
- Donghua Wang, Wen Yao, Tingsong Jiang, Xiaohu Zheng, Junqi Wu, and Xiaoqian Chen. Improving the Transferability of Adversarial Examples by Feature Augmentation. *arXiv*, 2024.
- Christopher Wiedeman and Ge Wang. Disrupting adversarial transferability in deep neural networks. *Patterns*, 3(5), 2022.
- Eric Wong and Zico Kolter. Provable defenses against adversarial examples via the convex outer adversarial polytope. In *International conference on machine learning*. PMLR, 2018.
- Huan Zhang, Hongge Chen, Chaowei Xiao, Sven Gowal, Robert Stanforth, Bo Li, Duane S. Boning, and Cho-Jui Hsieh. Towards stable and efficient training of verifiably robust neural networks. In *8th International Conference on Learning Representations, ICLR, 2020*.
- Shufei Zhang, Zhuang Qian, Kaizhu Huang, Qiufeng Wang, Rui Zhang, and Xinping Yi. Towards better robust generalization with shift consistency regularization. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139, pp. 12524–12534. PMLR, 2021.

APPENDIX

A GENERATION OF ADVERSARIAL EXAMPLES

This appendix first specifies the attack-generation procedure used throughout the paper, so that the contribution claims about mechanism are tied to a standard optimisation method. We use PGD, an iterative attack method (Madry et al., 2018). For untargeted attacks, PGD maximises the classifier loss $\mathcal{L}(f(\mathbf{x}'), y_{\text{true}})$; for targeted attacks, it minimises $\mathcal{L}(f(\mathbf{x}'), y_{\text{target}})$ according to:

$$\mathbf{x}'^{(k+1)} = \Pi_{\mathcal{S}} \left(\mathbf{x}'^{(k)} \pm \alpha \mathbf{g}_k \right) \quad (1)$$

Here $+$ denotes maximisation and $-$ denotes minimisation. $\mathbf{x}'^{(0)}$ is the initial perturbed input, α is the step size, $\mathbf{g}_k$ is the normalised gradient $\nabla_{\mathbf{x}} \mathcal{L}(f(\mathbf{x}'^{(k)}), y_{\text{class}})$, with y_{class} equal to y_{true} or y_{target} , and $\Pi_{\mathcal{S}}$ projects onto the perturbation set $\mathcal{S} = \{\mathbf{x}' \mid \|\mathbf{x}' - \mathbf{x}\|_p \leq \epsilon\}$ while clipping to the input domain. We use two norm constraints:

1. ℓ_{∞} : Constraint $\|\delta\|_{\ell_{\infty}} = \max_i |\delta_i| \leq \epsilon$. $\mathbf{g}_k$ is $\text{sign}(\nabla_{\mathbf{x}} \mathcal{L}(\cdot))$. $\Pi_{\mathcal{S}}$ clips each $\mathbf{x}'_i$ to $[\mathbf{x}_i - \epsilon, \mathbf{x}_i + \epsilon]$.
2. ℓ_2 : Constraint $\|\delta\|_{\ell_2} = \sqrt{\sum_i \delta_i^2} \leq \epsilon$. $\mathbf{g}_k$ is $\nabla_{\mathbf{x}} \mathcal{L}(\cdot) / \|\nabla_{\mathbf{x}} \mathcal{L}(\cdot)\|_{\ell_2}$. $\Pi_{\mathcal{S}}$ rescales $\delta = \mathbf{x}' - \mathbf{x}$ if $\|\delta\|_{\ell_2} > \epsilon$ via $\delta \leftarrow \epsilon \cdot \delta / \|\delta\|_{\ell_2}$.

B OMITTED PROOFS & THEORETICAL RESULTS

This section provides the formal support for the paper’s central contribution. We prove that, in the linear toy setting, optimal adversarial perturbations weight each input dimension according to how strongly its corresponding feature aligns with the direction toward the target decision boundary. The analysis uses the linear models without ReLU activation functions.

Let input $\mathbf{x} \in \mathbb{R}^d$ be encoded as $\phi(\mathbf{x}) = \mathbf{W}_e \mathbf{x}$ into latent representation $\mathbf{h} \in \mathbb{R}^m$, where the columns of $\mathbf{W}_e \in \mathbb{R}^{m \times d}$ form an overcomplete dictionary $\{\mathbf{v}_i\}_{i=1}^d$ with $m < d$. In the argmax task, the trained encoder-decoder pair empirically uses the decoder transpose, $\mathbf{W}_d = \mathbf{W}_e^{\top}$. Thus the class- j logit is $z_j = \mathbf{v}_j^{\top} \mathbf{h}$, where $\mathbf{v}_j$ is the j -th column of $\mathbf{W}_e$, and the prediction is $\hat{y} = \arg \max_j z_j$. Input perturbations become latent perturbations through $\Delta \mathbf{h} = \mathbf{W}_e \delta$, where $\delta = (\delta_1, \dots, \delta_d)^{\top}$ are perturbation coefficients.

The decision boundary between classes j and k is where their logits match: $\mathcal{B}_{jk} = \{\mathbf{h} : z_j = z_k\}$, equivalently $\{\mathbf{h} : (\mathbf{v}_k - \mathbf{v}_j)^{\top} \mathbf{h} = 0\}$. The vector $\mathbf{n} = (\mathbf{v}_k - \mathbf{v}_j)$ is the boundary normal and the direction that maximally increases the margin $z_k - z_j$. We restate the propositions before giving the proofs.

Proposition 1 (Optimal targeted attack). The optimal input perturbations δ that maximise movement from class j toward class k under constraint $\|\delta\|_2 = \epsilon$ satisfy:

$$\delta \propto \mathbf{W}_e^{\top} \mathbf{n}$$

Proof. To move a sample from being classified as j to being classified as k , we need to maximise the logit margin $z_k - z_j$. Under perturbation δ , the new margin becomes:

$$z'_k - z'_j = \mathbf{v}_k^{\top} (\mathbf{h} + \Delta \mathbf{h}) - \mathbf{v}_j^{\top} (\mathbf{h} + \Delta \mathbf{h}) \quad (2)$$

$$= (\mathbf{v}_k - \mathbf{v}_j)^{\top} (\mathbf{h} + \mathbf{W}_e \delta) \quad (3)$$

$$= (\mathbf{v}_k - \mathbf{v}_j)^{\top} \mathbf{h} + (\mathbf{v}_k - \mathbf{v}_j)^{\top} \mathbf{W}_e \delta \quad (4)$$

The change in margin is $(\mathbf{v}_k - \mathbf{v}_j)^{\top} \mathbf{W}_e \delta = \delta^{\top} \mathbf{W}_e^{\top} (\mathbf{v}_k - \mathbf{v}_j)$. We seek:

$$\max_{\delta} \delta^{\top} \mathbf{W}_e^{\top} (\mathbf{v}_k - \mathbf{v}_j) \quad \text{subject to} \quad \|\delta\|_2 = \epsilon$$

Let $\mathbf{g} = \mathbf{W}_e^{\top} (\mathbf{v}_k - \mathbf{v}_j)$. By the Cauchy-Schwarz inequality:

$$|\delta^{\top} \mathbf{g}| \leq \|\delta\|_2 \|\mathbf{g}\|_2 = \epsilon \|\mathbf{g}\|_2$$

Equality is achieved when δ and $\mathbf{g}$ are parallel. Given the constraint $\|\delta\|_2 = \epsilon$:

$$\delta = \frac{\epsilon}{\|\mathbf{W}_e^\top (\mathbf{v}_k - \mathbf{v}_j)\|_2} \mathbf{W}_e^\top (\mathbf{v}_k - \mathbf{v}_j)$$

□

Corollary (Interference drives vulnerability). For a targeted attack from class j to class k , the adversarial perturbation magnitude for input feature i is:

$$|\delta_i| \propto |\mathbf{v}_i^\top (\mathbf{v}_k - \mathbf{v}_j)|$$

where $\mathbf{v}_i$ is the i -th column of $\mathbf{W}_e$. This quantity represents the differential interference between feature i and the class representations.

This reveals the mechanism by which superposition creates adversarial vulnerability. Each input feature i is perturbed proportionally to $\mathbf{v}_i^\top (\mathbf{v}_k - \mathbf{v}_j)$ —the differential interference between feature i and the class representations. Under superposition, the non-orthogonality means that even semantically unrelated features have non-zero inner products with $(\mathbf{v}_k - \mathbf{v}_j)$, creating exploitable interference patterns. The multi-class setting amplifies this vulnerability, as with k classes there are multiple possible pairwise boundaries, each creating a distinct interference pattern. Adversarial perturbations leverage these cross-feature dependencies—they manipulate features that affect it through their interference with the class representations. This explains the vulnerability we observe empirically: attacks succeed not by directly increasing target class features, but by exploiting the web of interference created by superposition.

Prop. 1 characterises the optimal perturbation direction for moving from class j to k , providing the gradient for maximising the margin $z_k - z_j$. When intervening classes exist (where $z_j < z_i < z_k$ for some class i), following this gradient might cause the model to predict i before reaching target k . Iterative methods like PGD handle this by recomputing gradients at each step—the global attack path emerges from repeated local decisions rather than a single optimisation. Furthermore, while our analysis assumes $\mathbf{W}_d = \mathbf{W}_e^\top$ based on our empirical observations, the framework extends to arbitrary decoders. In the general case, with decoder $\mathbf{W}_d \in \mathbb{R}^{k \times m}$ having rows $\mathbf{w}_j^\top$, the optimal perturbation becomes $\delta \propto \mathbf{W}_e^\top (\mathbf{w}_k - \mathbf{w}_j)$, capturing interference between encoder directions and decoder weights.

Proposition 2 (Attack transferability). Consider encoders ϕ and ψ with basis matrices $\mathbf{W}_e \in \mathbb{R}^{m \times d}$ and $\mathbf{W}'_e \in \mathbb{R}^{m \times d}$ whose columns are related by orthogonal transformation $\mathbf{v}'_i = \mathbf{Q}\mathbf{v}_i$ (where $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$). If both models use their encoder transpose as decoder (i.e., $\mathbf{W}_d = \mathbf{W}_e^\top$ and $\mathbf{W}'_d = (\mathbf{W}'_e)^\top$), then both models have identical optimal input perturbation vectors for any targeted attack from class j to class k .

Proof. For model ϕ , the optimal perturbation from Prop. 1 is:

$$\delta^\phi \propto \mathbf{W}_e^\top (\mathbf{v}_k - \mathbf{v}_j)$$

For model ψ with transformed columns $\mathbf{v}'_i = \mathbf{Q}\mathbf{v}_i$:

$$\delta^\psi \propto (\mathbf{W}'_e)^\top (\mathbf{v}'_k - \mathbf{v}'_j) = (\mathbf{W}'_e)^\top (\mathbf{Q}\mathbf{v}_k - \mathbf{Q}\mathbf{v}_j)$$

Since $\mathbf{W}'_e = \mathbf{Q}\mathbf{W}_e$ (all columns are transformed), we have:

$$\delta^\psi \propto (\mathbf{Q}\mathbf{W}_e)^\top \mathbf{Q}(\mathbf{v}_k - \mathbf{v}_j) \tag{5}$$

$$= \mathbf{W}_e^\top \mathbf{Q}^\top \mathbf{Q}(\mathbf{v}_k - \mathbf{v}_j) \tag{6}$$

$$= \mathbf{W}_e^\top (\mathbf{v}_k - \mathbf{v}_j) \tag{7}$$

where the last equality uses $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$. Thus δ^ϕ and δ^ψ have identical proportionality. Under the same norm constraint $\|\delta\|_2 = \epsilon$, we have $\delta^\phi = \delta^\psi$. □

Prop. 2 is used to explain why attacks transfer between models with similar training regimes. When models learn feature representations that differ only by orthogonal transformation—essentially the

same geometric structure in different orientations—they share identical vulnerability patterns in input space. The orthogonal transformation preserves all inner products between features, maintaining the interference patterns that attacks exploit. This provides an explanation for our empirical observation that models trained on data with the same correlations exhibit high attack transferability: they discover similar feature geometries up to rotation, leading to shared adversarial vulnerabilities.

C TOY MODEL EXPERIMENTS

This section provides supplementary details, extended results and further discussion for the toy model experiments discussed in the main paper. We present model accuracies across a wider range of parameters than shown in the main text, offering insight into how model capacity and data characteristics like sparsity influence the learning process and the conditions under which feature superposition appears. Subsequently, we offer additional visual examples that correspond to Fig. 1, illustrating the mechanics of adversarial attacks under various conditions.

C.1 HYPOTHESES TESTING FRAMEWORK

We explicitly state our hypotheses for the three research questions in Sec. 3.

Research Q1: Do adversarial perturbations exploit superposition geometry?

- H_0 : Adversarial perturbations are random with respect to feature geometry.
- H_1 : Adversarial perturbations systematically exploit geometric relationships between superposed representations.

Research Q2: Do data correlations determine superposition geometry?

- H_0 : Input correlations have no systematic effect on learned geometries.
- H_1 : Input correlations determine geometric arrangements across model initializations.

Research Q3: Does shared geometry explain attack transferability?

- H_0 : Attack transferability is independent of geometric similarity.
- H_1 : Transferability increases with shared latent structure.

We test these hypotheses through controlled experiments:

- $H_1(1)$: We measure the input perturbation profile alignment with a class’s latent representation and the latent attack vector.
- $H_1(2)$: We systematically vary input correlations and measure resulting geometries.
- $H_1(3)$: We quantify transferability rates across models with varying geometric similarity.

C.2 ACCURACY OF TOY MODEL FOR A RANGE OF PARAMETERS

The toy model experiments presented in the main paper predominantly used low-dimensionality settings for conceptual clarity. To demonstrate the model’s behaviour more broadly, this subsection details the classification accuracies achieved by the CE toy model. These results are presented across varying hidden layer size (h), number of classes (k), number of features, and levels of sparsity (S), to provide insight into when the models learn to represent features in superposition. The sparsity level represents the probability that any individual input feature $x_i^{(j)}$ is set to zero, with higher values of S indicating greater input sparsity. We report results as a function of feature density $1 - S$, the probability that a feature is non-zero. Tab. 2 and Tab. 3 provide context on the model’s performance limits and its ability to learn latent representations in superposition.

C.3 CORRELATIONS CONSTRAIN ARRANGEMENTS

Tab. 4 extends the main-text contribution that input correlations constrain latent geometry. It reports the same geometric-similarity analysis across a wider range of classes (k) and hidden dimensions (m).

Table 2: Toy-model accuracy under fixed bottleneck dimension ($m = 2$), varying number of classes (k), total input features (features = $k \times 3$), and input feature densities ($1 - S$). The table supports the contribution setup by showing how sparsity affects performance as more classes are superposed in a constrained latent space.

Classes (k)	Features	Hidden (m)	Accuracy at Input Feature Density ($1 - S$)							
			1.0	0.57	0.33	0.19	0.11	0.06	0.04	0.02
3	9	2	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
4	12	2	0.67	0.71	0.82	0.92	0.98	0.99	1.00	1.00
5	15	2	0.53	0.50	0.65	0.77	0.89	0.95	0.98	0.99
6	18	2	0.40	0.43	0.51	0.66	0.82	0.93	0.97	0.99
7	21	2	0.34	0.34	0.40	0.53	0.73	0.87	0.95	0.98
8	24	2	0.30	0.30	0.33	0.40	0.63	0.82	0.93	0.97
9	27	2	0.24	0.26	0.30	0.35	0.57	0.75	0.89	0.96
10	30	2	0.22	0.24	0.26	0.31	0.50	0.72	0.87	0.95
15	45	2	0.14	0.15	0.16	0.17	0.25	0.40	0.65	0.86
20	60	2	0.10	0.10	0.11	0.13	0.15	0.24	0.44	0.76
25	75	2	0.07	0.08	0.09	0.10	0.12	0.16	0.26	0.62
30	90	2	0.06	0.07	0.07	0.07	0.09	0.11	0.21	0.45

Table 3: Toy-model accuracy across numbers of classes (k), total input features, bottleneck dimensions (m), and sparsity levels (S), with different numbers of features per class (p). These controls contextualise the superposition-pressure experiments used in the main text.

Classes (k)	Features	Hidden (m)	Accuracy at Input Feature Density ($1 - S$)							
			1.0	0.57	0.33	0.19	0.11	0.06	0.04	0.02
30	30	30	0.23	0.24	0.38	0.62	0.83	0.94	0.99	1.00
30	90	90	0.27	0.27	0.33	0.41	0.51	0.67	0.85	0.94
40	40	30	0.67	0.54	0.73	0.77	0.88	0.96	0.99	0.99
40	120	30	0.71	0.64	0.65	0.72	0.73	0.79	0.89	0.96
60	60	10	0.05	0.07	0.12	0.25	0.47	0.73	0.90	0.97
60	180	10	0.08	0.10	0.13	0.17	0.22	0.32	0.52	0.75
80	80	30	0.15	0.17	0.23	0.41	0.63	0.79	0.91	0.98
80	240	30	0.23	0.22	0.31	0.41	0.48	0.53	0.66	0.80
100	100	10	0.03	0.04	0.05	0.10	0.21	0.43	0.69	0.87
100	300	10	0.04	0.05	0.06	0.09	0.12	0.15	0.25	0.45

Table 4: Geometric similarity across correlation types and superposition configurations, extending the main-text claim that stronger correlation structure constrains learned feature geometry.

Correlation Type	k	m	Geometric Similarity
Uncorrelated	6	2	0.18 ± 0.15
	30	10	0.17 ± 0.02
	90	30	0.17 ± 0.01
Paired	6	2	0.47 ± 0.07
	30	10	0.26 ± 0.07
	90	30	0.25 ± 0.01
Global	6	2	0.92 ± 0.04
	30	10	0.88 ± 0.01
	90	30	0.80 ± 0.01

C.4 HOW THE TOY MODEL MAPS TO REAL-WORLD MODELS

This subsection explains how the toy-model contribution should be interpreted when moving toward real-world models. Larger models often show somewhat greater adversarial robustness, but the effect is weak; with adversarial training, scaling trends are clearer, yet robustness gains remain largely specific to the attack type used during training rather than becoming fully general (Howe et al., 2025).

Two scaling forces are relevant. Larger models have more capacity for representing concepts, but broad tasks such as next-token prediction also contain a long tail of useful concepts to encode. As a result, the pressure that drives superposition may persist even as models grow; superposition has been observed in frontier-scale settings (Lindsey et al., 2025). Dictionary-learning evidence points in the same direction: SAEs require larger dictionaries for larger models (Gao et al., 2025), suggesting that the number of represented features scales with model size. Thus the compression–interference tension highlighted in the main paper need not disappear at scale.

Because both superposition and adversarial vulnerability persist in larger models, the mechanism studied here remains relevant beyond the toy setting. A key future direction is to characterise how superposition geometry changes with scale and whether those changes can be used to reduce vulnerability.

C.4.1 SEPARATING SUPERPOSITION EFFECTS FROM CAPACITY REDUCTION

To separate superposition pressure from capacity reduction, we hold the bottleneck dimension fixed and vary only the number of classes. Specifically, we fix $m = 2$ and vary k , then report model accuracy, robust accuracy, and transferability in Tab. 5. These controls support the main contribution by showing that increased superposition pressure (k/m), independent of bottleneck capacity, drives the observed vulnerabilities.

C.4.2 ADDITIONAL EXAMPLES OF AEXS IN TOY MODEL

Fig. 1 shows the main visual example of attacks exploiting interference between superposed latent features. Here, Fig. 5 provides additional examples to make the mechanism easier to inspect across attack norms and bottleneck dimensions:

- Fig. 5a shows an additional instance of the setup in Sec. 3.1 ($m = 2, k = 7$) with an ℓ_2 -norm PGD attack, demonstrating the IPP and latent space manipulations that lead to misclassification.
- Fig. 5b shows a similar setup ($m = 2, k = 7$) using an ℓ_∞ -norm PGD attack on a less sparse input.
- Fig. 5c shows an example with increased bottleneck dimensionality ($m = 3, k = 7$) and accompanying ℓ_2 -norm PGD attack. The feature vector similarity matrix used to calculate geometric similarity is also shown.

D CIFAR-10 EXPERIMENTS

The scale experiment in Sec. 4 tests whether the toy-model contribution extends to CIFAR-10 (Krizhevsky, 2009) models with engineered bottlenecks. This appendix gives the architectural and training details needed to reproduce that scale test and reports extended robustness and transferability results.

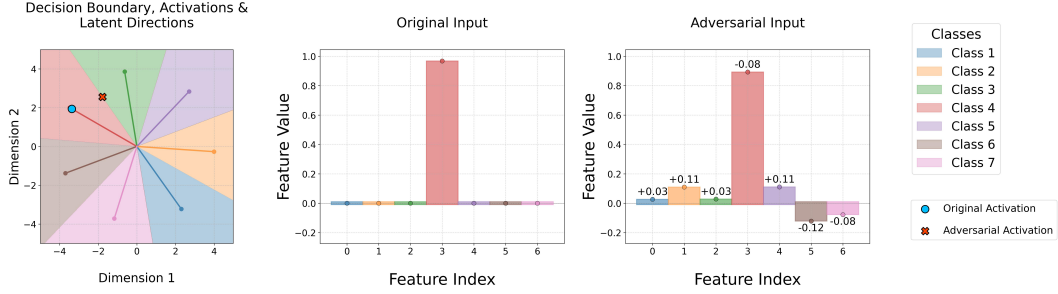
D.1 ARCHITECTURE & TRAINING INFORMATION

The base ViT (Dosovitskiy et al., 2020) used 6 transformer layers, embedding dimension (d) 512, and 8 attention heads per layer. CIFAR-10 images of size 32×32 were split into 4×4 patches. Each transformer-block Multilayer Perceptron (MLP) had hidden dimension 512, and the model used learned positional embeddings.

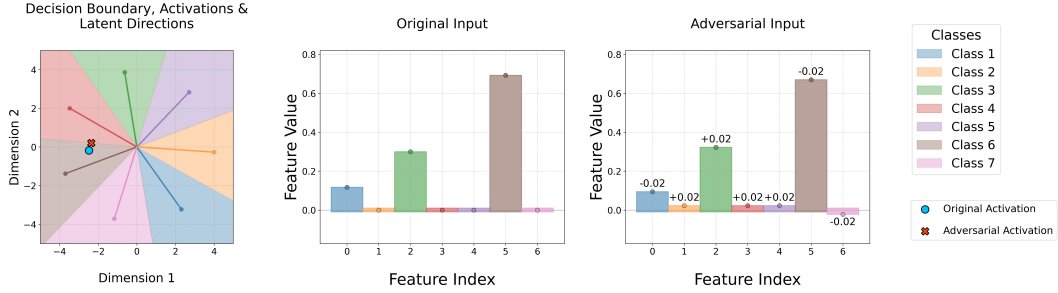
The bottleneck used to induce class-feature superposition consisted of a linear encoder followed by a linear decoder. The encoder projected pre-classification ViT activations into an m -dimensional latent space, with $m \in \{2, 3, 5, 10\}$, and the decoder mapped those representations back to the $k = 10$ CIFAR-10 logits.

Table 5: Performance metrics across correlation types, perturbation budgets (ε), and number of classes (k), isolating the effect of superposition pressure at fixed bottleneck dimension.

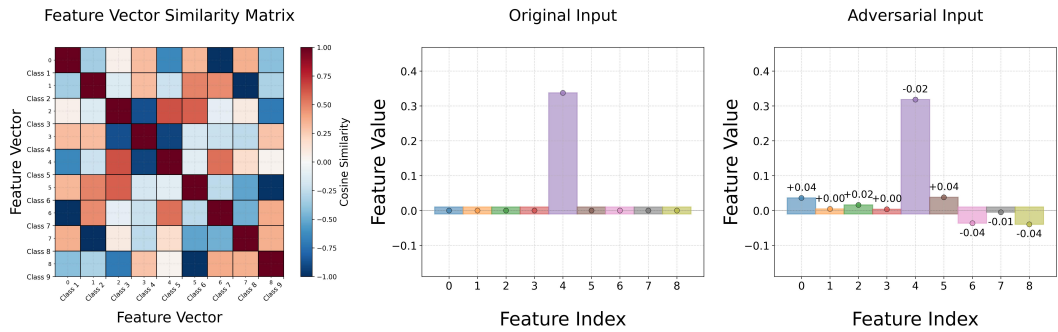
Correlation Type	Parameters		Performance (%)		
	ε	k	Accuracy	Robust Acc.	Attack Trans.
Uncorrelated	0.05	2	100.0	100.0	0.0
		4	100.0	94.9	21.2
		6	99.9	87.0	9.8
		8	99.8	77.7	17.5
	0.1	2	100.0	100.0	0.0
		4	100.0	88.1	9.8
		6	99.7	75.4	9.9
		8	99.6	56.3	23.5
	0.5	2	100.0	100.0	0.0
		4	100.0	99.4	0.0
		6	99.9	62.9	30.5
		8	99.5	24.9	39.0
Fully Correlated	0.05	2	100.0	100.0	0.0
		4	100.0	95.8	98.4
		6	97.3	70.6	98.4
		8	93.0	57.4	97.2
	0.1	2	100.0	100.0	0.0
		4	100.0	88.7	98.5
		6	97.7	45.6	99.7
		8	94.2	34.8	98.6
	0.5	2	100.0	100.0	0.0
		4	100.0	90.0	100.0
		6	97.2	34.6	100.0
		8	93.0	32.0	100.0



(a) An ℓ_2 -norm attack changing the classification of an input of class 4 to class 3. The left plot shows original and adversarial activations in latent space, along with representation directions. The right plots show original and perturbed input feature values.



(b) An ℓ_∞ -norm attack changing the classification of an input of class 6 to class 4. The left plot shows original and adversarial activations in latent space relative to class latent directions. The right plots show original and perturbed input feature values.



(c) An ℓ_2 -norm adversarial attack in a 7-class setup with an increased bottleneck dimension $m = 3$. The leftmost plot now shows the cosine similarity matrix between pairs of latent representations for each of the classes.

Figure 5: Additional toy-model AExs illustrating the paper’s mechanism across varied conditions: perturbations alter input features in patterns that correspond to movements through activation space and superposed class geometry.

The base ViT was trained on CIFAR-10 for 250 epochs using CE loss, Adam with default PyTorch parameters, learning rate 0.001, batch size 512, cosine annealing, and dropout. We trained five random seeds to measure variability. After base-model training, the ViT weights were frozen and the bottleneck was trained for 30 epochs with learning rate 0.001. Preprocessing used RandomCrop with 4-pixel padding, resizing to the target size (32x32 by default), and RandomHorizontalFlip augmentation.

D.1.1 NORMALISED ROBUST ACCURACY ACROSS PERTURBATION MAGNITUDES

Fig. 4 (right) reports normalised robust accuracy for one perturbation budget. This subsection broadens that result across attack strengths to show that the contribution pattern is not tied to a single ϵ . We report both ℓ_2 -norm (Tab. 7) and ℓ_∞ -norm (Tab. 6) PGD attacks, giving a fuller view of how superposition pressure and attack strength jointly affect robustness.

For these evaluations, AExs were generated with 100-step PGD using step size (α) 0.01. Robust accuracy was evaluated on 500 samples per configuration, and we report the mean normalised robust accuracy with standard deviation across five seeds.

Table 6: Mean normalised robust accuracy ($\pm$ standard deviation across 5 seeds) for CIFAR-10 ViT models with different bottleneck dimensions (m) under ℓ_∞ -norm PGD attacks with varying ϵ . Robust accuracy is normalised by each bottlenecked model’s clean accuracy, and the trend supports the scale contribution in Sec. 4: stronger superposition pressure yields lower robustness across ϵ .

ϵ	Bottleneck Dimension (m)			
	2	3	5	10
0.001	96.0% $\pm$ 0.4%	97.4% $\pm$ 0.7%	97.9% $\pm$ 0.4%	98.1% $\pm$ 0.3%
0.01	61.7% $\pm$ 3.6%	69.6% $\pm$ 3.5%	77.0% $\pm$ 0.9%	81.8% $\pm$ 3.2%
0.05	4.9% $\pm$ 1.5%	6.7% $\pm$ 1.6%	9.3% $\pm$ 0.6%	10.5% $\pm$ 0.8%
0.1	0.1% $\pm$ 0.2%	0.2% $\pm$ 0.3%	0.2% $\pm$ 0.4%	0.2% $\pm$ 0.4%
0.5	0.0% $\pm$ 0.0%	0.0% $\pm$ 0.0%	0.0% $\pm$ 0.0%	0.0% $\pm$ 0.0%

Table 7: Mean normalised robust accuracy ($\pm$ standard deviation across 5 seeds) for CIFAR-10 ViT models with different bottleneck dimensions (m) under ℓ_2 -norm PGD attacks with varying ϵ . Robust accuracy is normalised by each bottlenecked model’s clean accuracy; smaller m and larger ϵ produce the lower-robustness pattern discussed in the scale experiment.

ϵ	Bottleneck Dimension (m)			
	2	3	5	10
0.1	90.4% $\pm$ 1.7%	91.8% $\pm$ 0.7%	94.6% $\pm$ 1.2%	95.0% $\pm$ 0.7%
0.5	58.5% $\pm$ 5.0%	60.8% $\pm$ 5.0%	69.2% $\pm$ 1.8%	72.6% $\pm$ 2.8%
1.0	41.7% $\pm$ 4.8%	44.0% $\pm$ 3.0%	50.4% $\pm$ 0.6%	54.9% $\pm$ 1.7%
2.0	34.0% $\pm$ 4.6%	36.2% $\pm$ 3.3%	41.5% $\pm$ 1.3%	47.4% $\pm$ 2.4%
5.0	30.2% $\pm$ 4.1%	32.9% $\pm$ 3.5%	37.3% $\pm$ 1.9%	42.7% $\pm$ 1.0%

D.1.2 ATTACK TRANSFERABILITY ACROSS PERTURBATION MAGNITUDES

We also report attack transferability across perturbation magnitudes (ϵ) and bottleneck dimensions (m), extending the main-text claim that stronger superposition increases shared attack success. Tab. 9 gives ℓ_2 -norm transferability, and Tab. 8 gives ℓ_∞ -norm transferability.

D.2 CORRELATIONS IN CLASS FEATURES

In Sec. 3, input correlations drove superposition arrangements. In the CIFAR-10 bottleneck, the analogous driver is not raw class correlation but correlation in the representations reaching the classifier. At the classification layer, this structure is closely related to the confusion matrix: classes that cluster are those the network finds similar and tends to confuse. To test this interpretation, we repeat the experiment by fine-tuning timm/vit_base_patch16_384, which was pretrained on ImageNet-21k (14 million images, 21,843 classes) and ImageNet (1 million images, 1,000 classes). After CIFAR-10 fine-tuning and the same bottleneck training as in Sec. 4, the resulting geometry shows random ordering across initialisations with approximately equal feature spacing (*i.e.* neural collapse Kothapalli (2023)). In this case, performance is near 100%, the confusion matrix is close to the identity, and the superposed geometry loses its structured clustering. By contrast, models trained only on CIFAR-10 converge to similar geometries because they share a learned difficulty structure: the same class pairs are hard to distinguish, producing consistent superposition patterns.

Table 8: Attack transferability (%) for ℓ_∞ -norm PGD attacks on CIFAR-10 ViT models. Each row fixes ϵ and bottleneck dimension m ; slash-separated entries show transfer from the listed source seed to three target seeds, and ‘Mean $\pm$ Std’ averages all 12 source-target pairings. The table supports the scale contribution: stronger superposition yields more consistent geometry and higher transferability.

ϵ	m	Seed 10 vs. Seeds 20/30/40	Seed 20 vs. Seeds 10/30/40	Seed 30 vs. Seeds 10/20/40	Seed 40 vs. Seeds 10/20/30	Mean $\pm$ Std
0.001	2	77.8/66.7/44.4	25.0/62.5/25.0	72.7/72.7/72.7	58.3/75.0/58.3	59.3 $\pm$ 17.7
	3	57.1/28.6/28.6	14.3/28.6/28.6	63.6/72.7/72.7	42.9/42.9/42.9	43.6 $\pm$ 18.4
	5	70.0/50.0/50.0	57.1/71.4/57.1	50.0/25.0/25.0	16.7/50.0/33.3	46.3 $\pm$ 16.9
	10	55.6/55.6/66.7	12.5/37.5/25.0	16.7/33.3/16.7	28.6/71.4/28.6	37.3 $\pm$ 19.3
0.01	2	60.3/52.6/48.7	63.8/42.0/60.9	53.8/40.9/52.7	51.2/55.8/46.5	52.4 $\pm$ 6.9
	3	42.9/46.4/50.0	46.4/42.9/49.1	44.2/38.9/44.2	42.9/45.1/41.8	44.6 $\pm$ 3.0
	5	43.3/41.1/43.3	39.2/30.4/41.8	46.0/40.2/47.1	41.2/35.3/32.9	40.2 $\pm$ 4.8
	10	42.5/46.0/44.8	38.0/39.4/46.5	43.5/47.8/47.8	32.6/40.0/35.8	42.1 $\pm$ 4.7
0.05	2	35.0/36.4/37.8	38.8/36.0/46.3	37.4/31.3/41.7	42.0/42.0/40.7	38.8 $\pm$ 3.8
	3	29.9/29.9/37.1	30.4/30.1/35.6	30.2/30.2/35.1	30.1/31.9/34.4	32.1 $\pm$ 2.6
	5	26.8/25.3/29.2	25.2/21.5/26.4	28.0/28.0/26.5	26.0/26.6/21.9	25.9 $\pm$ 2.2
	10	27.1/24.9/30.4	21.6/23.0/28.4	25.3/28.1/30.6	21.0/24.9/20.1	25.4 $\pm$ 3.4
0.1	2	34.2/35.6/38.2	36.7/34.9/44.5	37.9/30.0/40.8	41.2/42.0/40.3	38.0 $\pm$ 3.8
	3	29.6/28.9/35.2	28.6/29.2/34.8	29.1/28.2/34.0	28.8/31.1/32.8	30.9 $\pm$ 2.5
	5	25.4/23.3/27.0	23.8/19.6/24.6	25.9/26.7/25.4	24.6/24.3/20.7	24.3 $\pm$ 2.1
	10	24.8/22.6/29.7	20.1/21.6/26.1	22.5/25.5/28.0	19.5/23.3/18.7	23.5 $\pm$ 3.3

D.2.1 RESNET-18

To test architectural generality, we repeat the bottleneck experiment with a ResNet-18 (He et al., 2016) base model instead of a ViT. The ResNet-18 reaches 92% clean accuracy, slightly above the 89% ViT comparison. Robust accuracy falls slightly faster for ResNet-18 at matched ϵ values, but the contribution pattern is preserved: increasing bottleneck pressure decreases normalised robust accuracy and increases transferability.

E MODULAR ADDITION

The modular-addition experiment in Sec. 5 defines the boundary of the superposition claim: adversarial vulnerability can also arise when relevant features are orthogonal. This appendix gives the detailed setup, additional results, and supporting discussion for that algorithmic-brittleness contribution.

Table 9: Attack transferability (%) for ℓ_2 -norm PGD attacks on CIFAR-10 ViT models. The format mirrors Tab. 8; these ℓ_2 results further support the scale claim in Sec. 4 that stronger superposition increases attack transferability.

ϵ	m	Seed 10 vs. Seeds 20/30/40	Seed 20 vs. Seeds 10/30/40	Seed 30 vs. Seeds 10/20/40	Seed 40 vs. Seeds 10/20/30	Mean $\pm$ Std
0.1	2	70.6/64.7/41.2	60.0/50.0/60.0	64.3/60.7/57.1	48.1/63.0/51.9	57.6 ± 8.0
	3	39.1/39.1/47.8	50.0/60.7/50.0	48.1/55.6/51.9	48.0/28.0/40.0	46.5 ± 8.4
	5	43.5/34.8/43.5	31.6/31.6/42.1	42.9/35.7/21.4	30.4/39.1/34.8	35.9 ± 6.4
	10	45.5/59.1/45.5	19.0/28.6/28.6	37.5/43.8/43.8	23.8/52.4/38.1	38.8 ± 11.4
0.5	2	59.6/52.1/48.9	60.8/35.4/60.8	54.1/39.4/51.4	43.8/49.5/46.7	50.2 ± 7.7
	3	38.7/40.6/48.1	36.1/36.8/41.7	37.4/36.5/43.5	34.4/34.4/36.7	38.7 ± 3.9
	5	38.4/32.0/37.6	29.4/25.7/32.1	39.1/33.9/38.3	31.8/29.9/32.7	33.4 ± 4.0
	10	38.3/37.4/37.4	26.5/28.4/31.4	32.4/35.3/35.3	23.4/28.2/25.0	31.6 ± 5.0
1.0	2	50.8/43.7/44.4	51.7/36.4/50.0	46.1/32.9/44.7	42.7/46.7/45.3	44.6 ± 5.3
	3	35.7/36.9/41.7	31.8/36.4/42.6	33.7/32.0/37.3	34.9/32.0/35.5	35.9 ± 3.3
	5	32.1/28.9/33.2	30.9/23.0/30.9	35.9/31.5/33.7	27.7/28.2/27.1	30.3 ± 3.3
	10	32.3/32.3/38.5	23.7/27.7/29.9	29.1/32.0/34.3	22.2/25.6/21.1	29.1 ± 5.0
2.0	2	47.0/41.6/44.3	48.2/36.2/48.2	43.9/33.5/42.1	43.4/43.4/42.8	42.9 ± 4.2
	3	31.9/37.7/40.3	31.1/35.6/40.6	32.6/31.1/36.8	35.4/33.3/34.3	35.1 ± 3.2
	5	29.9/25.4/32.1	26.3/22.1/26.7	32.3/31.8/29.5	27.9/27.9/23.6	28.0 ± 3.2
	10	30.8/30.4/35.7	22.5/24.9/28.6	24.8/29.7/32.7	20.4/26.1/19.0	27.1 ± 4.9
5.0	2	44.7/46.7/40.8	45.3/37.3/48.7	42.7/33.9/41.5	42.3/42.3/40.6	42.2 ± 3.8
	3	32.3/35.8/40.8	31.3/32.6/39.9	31.0/30.0/33.8	35.0/33.0/33.0	34.1 ± 3.2
	5	30.6/24.1/28.6	28.3/20.9/24.8	31.7/31.3/27.4	28.1/29.0/23.2	27.3 ± 3.3
	10	29.8/27.4/32.7	19.7/24.1/28.9	24.9/29.9/31.2	18.8/25.8/19.2	26.0 ± 4.6

E.1 LEARNED ALGORITHMIC REPRESENTATION IN MODULAR ADDITION

As detailed in Nanda et al. (2023) and Prieto et al. (2025b), the network solves modular addition by discovering a trigonometric algorithm. The encoder projects inputs onto non-basis-aligned linear representations of $\sin(\omega_k a)$ and $\cos(\omega_k a)$ at key frequencies $\omega_k = 2\pi k/P$. These features are orthogonal, *i.e.* not in superposition, and their sinusoidal values are linearly probeable. The MLP then uses sum-angle identities to compute $\cos((a + b)\omega_k)$ for multiple k , whose contributions constructively interfere at the logits to select the correct class. We generate AExs by relaxing the one-hot constraint and perturbing input $\mathbf{x}_j \in \mathbb{R}^P$ (encoding integer j) to $\mathbf{x}' = \mathbf{x} + \delta$ using ℓ_2 - and ℓ_∞ -norm PGD. For simplicity, only a is attacked while b is fixed.

As explained in Nanda et al. (2023), the learned algorithm has a specific representation-level vulnerability. W_e encodes each integer’s sine and cosine values at the key frequencies, and these values are linearly decodable. Scaling the corresponding feature vector changes the encoded sinusoidal value itself, rather than merely changing activation intensity. Integer embeddings W_e^j project onto a Fourier-like basis, creating a circular or periodic embedding structure. Unlike the superposition arrangements driven by data correlations, this circularity comes from the sine/cosine encoding of non-correlated input integers.

The model uses approximately seven key frequencies (ω_k). Each embedding W_e^j for input integer x_j primarily encodes fourteen trigonometric features, $\cos(\omega_i x_j)$ and $\sin(\omega_i x_j)$ for $i \in \{1, 2, \dots, 7\}$. The concatenated 200-dim. vector therefore supplies roughly 28 key trigonometric features to the MLP, fourteen for a and fourteen for b . The MLP combines them via sum-angle identities (e.g., $\cos(A + B) = \cos A \cos B - \sin A \sin B$) to compute $(x_1 + x_2) \pmod{P}$. This algorithmic

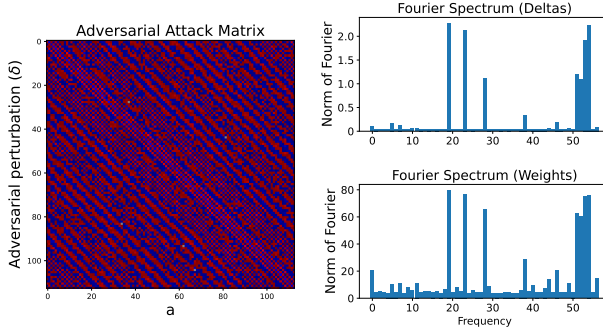


Figure 6: Mechanistic signature of modular-addition attacks. Left: each column is the mean attack δ added to a to successfully attack $a + b \bmod p$. Right: Fourier-component norms in δ and $W_{MLP_{out}}$ highlight the learned key frequencies.

Table 10: Minimum ℓ_2 -norm budget required to successfully attack $0 + 0 \bmod p$, comparing mechanism-informed attacks against PGD and naive baselines.

Attack Method	Success ϵ
PGD	0.17
Informed Attack	0.22
Single a	0.97
ℓ_∞ Noise	2.04
Uniform δ	6.20

brittleness arises because adversarial perturbations corrupt precisely these learned trigonometric features.

E.2 EXPERIMENT DETAILS FOR MODULAR ADDITION TASK

The modular-addition model is trained for addition modulo 113. Inputs are integer pairs (a, b) with $a, b \in \{0, \dots, 112\}$. Each integer maps to a 100-dimensional vector through a shared embedding $\mathbf{W}_E \in \mathbb{R}^{113 \times 100}$, and $\mathbf{W}_E^a$ and $\mathbf{W}_E^b$ are concatenated into a 200-dim. feature vector. A 3-hidden-layer MLP with 200 ReLU neurons per layer maps this vector to 113 logits. The model is trained end-to-end with cross-entropy loss and AdamW (weight decay 4), reaching 100% test accuracy. AExs are generated with PGD under ℓ_2 and ℓ_∞ norms for varying ϵ , perturbing $\mathbf{W}_E^a$ while holding b fixed so that $(a + b) \bmod 113$ is misclassified.

E.3 ALGORITHMIC BRITTLINESS

Frequency-aligned perturbations cause misclassification by altering the sinusoidal values read from the encoder features. A neuron-aligned idealisation makes the mechanism clear: $\mathbf{W}_e$ would contain a row for the sine or cosine of each ω_k , and the element at column $\mathbf{w}_e^a$ would contain $\sin(\omega_k a)$ or $\cos(\omega_k a)$. For input a , the one-hot $\mathbf{x}_a$ selects $\mathbf{w}_e^a$, giving $\mathbf{h}_a = \mathbf{w}_e^a$. The perturbation δ turns $\mathbf{x}_a$ into a dense vector $\mathbf{x}'_a$, so $\mathbf{h}'_a = \mathbf{W}_e \mathbf{x}'_a = (1 + \epsilon_a) \mathbf{h}_a + \sum_{k \neq a} \epsilon_k \mathbf{w}_e^k$. The added $k \neq a$ sinusoidal terms corrupt the values used by the MLP and produce misclassification. Because the logits depend on constructive interference across each ω_k , disrupting one ω_k is not enough; effective attacks must choose δ to alter multiple frequency components together.

E.4 ANALYTIC CONSTRUCTION OF ATTACKS AND BASELINE COMPARISONS

The main paper uses the learned trigonometric algorithm to construct analytic adversarial perturbations. Sec. 5 benchmarks this ‘informed attack’ against PGD and naive baselines; here we specify those baselines:

- **PGD Attack:** Standard PGD used to find the adversary.
- **Informed Attack:** The analytically constructed perturbation, from combinations of cosines.
- **Single a :** This involved finding the minimal attack to a single input to cause misclassification.
- **ℓ_∞ Noise:** This baseline involved adding random noise δ_{noise} to the input a . Each component of δ_{noise} was sampled independently and uniformly from $[-\epsilon, \epsilon]$. The ϵ that successfully induced a misclassification is reported.
- **Uniform δ :** The perturbation vector δ is constrained to have all components equal, i.e., $\delta_j = c \cdot 1$, where c is a scalar. The magnitude of c is reported such that adding δ to the input causes a misclassification.

E.5 DETAILS ON CERTIFIED ROBUST TRAINING

To test whether a defence changes the identified mechanism, we perform certified robust training against ℓ_∞ -norm perturbations. We use RSIP-IBP (Zhang et al., 2020; Henriksen & Lomuscio, 2023), an abstraction-based method in the family of certified defences (Wong & Kolter, 2018; Raghunathan et al., 2018; Palma et al., 2022). RSIP-IBP combines Interval Bound Propagation (IBP) (Katz et al., 2017; Ehlers, 2017) with Reversed Symbolic Interval Propagation (RSIP) to compute reachable output sets for network layers and certify robustness within a perturbation budget. The training loss includes both a standard loss and an RSIP loss. Certified training runs for 1000 epochs, with the first 200 epochs as warm-up, learning rate 1×10^{-3} , and weight decay 1×10^{-4} . We use the same train/test splits as standard training. Tab. 11 summarises certified training across training budgets (ϵ_{train}) and attack budgets (ϵ_{attack}).

Table 11: Verified robust accuracy (%) for modular-addition models under ℓ_∞ -norm perturbations. The table compares the standard model with RSIP-IBP-trained models at different training budgets (ϵ_{train}), evaluated against attack budgets (ϵ_{attack}). The results support the modular-addition contribution: certified training improves robustness within specified budgets, but higher-budget attacks can still exploit the same algorithmic sensitivities.

Attack Epsilon (ϵ_{attack})	Train Epsilon (ϵ_{train})	Standard Acc. (Pre-Rob., %)	Robust Acc. (Pre-Rob., %)	Standard Acc. (Robust, %)	Robust Acc. (Robust, %)
5.0×10^{-4}	10^{-4}	100.0	100.0	100.0	100.0
	10^{-3}				100.0
1.0×10^{-3}	10^{-4}	100.0	76.0	100.0	93.8
	10^{-3}				100.0
2.0×10^{-3}	10^{-4}	100.0	0.0	100.0	0.0
	10^{-3}				95.7
5.0×10^{-3}	10^{-4}	100.0	0.0	100.0	0.0
	10^{-3}				85.3

As discussed in Sec. 5, subsequent PGD attacks on the robustly trained models retained the same mechanism: successful perturbations still targeted the key frequency components of the learned algorithm. Certified training therefore mainly increased the perturbation magnitude required for misclassification at certified budgets, rather than changing the attack vector’s alignment with the model’s algorithmic sensitivities.

E.6 ON A POTENTIAL RECONCILIATION WITH SUPERPOSITION VULNERABILITY

Although this vulnerability arises without feature superposition, it resembles superposition-driven failures in one important respect: both depend on sparsity assumptions in the input representation. This motivates a *reconciliation conjecture*. Algorithmic vulnerabilities may persist in orthogonal circuits when upstream sparsity assumptions are violated. If earlier layers produce noisy or non-sparse features, perhaps through adversarial interference, those deviations can propagate into downstream circuits that are otherwise well isolated. A circuit that works on clean sparse inputs may then fail because its received input is effectively already perturbed. This gives a plausible route by which frequency-disruption mechanisms like the one studied here could be activated even without local superposition. The interaction between upstream superposition, downstream algorithmic brittleness, and layer-wise vulnerability mechanisms remains underexplored; we leave detailed empirical and theoretical study to future work.

F INSPECTING ADVERSARIAL PERTURBATIONS VIA SAE FEATURES

The main paper studies settings with explicit class features (toy model) or class-level bottleneck representations (ViT experiments). In large practical networks, however, interference may involve unlabelled latent features across many layers. As noted in the future-work discussion, SAEs offer

a promising unsupervised route to extracting more linear features from NNs (Bricken et al., 2023). This appendix presents a preliminary feasibility study: we use SAE-extracted features to inspect how a large-scale ViT responds internally to AExs versus clean inputs. These results are exploratory and support future work on extending the contribution mechanism to large models, rather than serving as a new conclusive claim. We outline the setup, validate SAEs on adversarial inputs, and then compare layer-wise feature differences.

F.1 EXPERIMENT SETUP

We analyse SAE-extracted features from the vision encoder of LAION/CLIP-ViT-B-32-DataComp.XL-s13B-b90K, a 12-layer ViT with $\approx 70\%$ zero-shot accuracy on ImageNet-1k Deng et al. (2009). We use open-source Prisma SAEs that map the 768-dimensional activation space to 49,152 dictionary features ($64\times$ expansion) (Joseph et al., 2025). Across all layers, we examine SAEs trained on either mean patch-token representations or CLS-token representations. We use a high activation threshold (0.1) to focus on the most active features and a low threshold (0.001) to include weaker activations. The comparison uses 10,000 original/successfully attacked image pairs (ℓ_∞ -norm, $\epsilon = 0.02$), providing an initial view of how adversarial perturbations appear in SAE feature space.

F.2 SAE VALIDITY FOR ADVERSARIAL INPUTS

Given that AExs are out-of-distribution inputs, and our aim is to use SAE (standardly trained) to understand them, we first conduct a basic validation. This subsection examines the impact on model accuracy and SAE reconstruction when SAEs are inserted at inference time for both clean and adversarial inputs. We are investigating whether the SAEs process features relevant to the classification task, even under adversarial conditions, before attempting to interpret differences in feature activations.

Table 12: SAE insertion sanity check for ImageNet-1k classification accuracy. A pre-trained SAE is inserted one layer at a time during inference for original, ℓ_∞ adversarial ($\epsilon = 0.02$), and random noisy inputs. ‘Accuracy’ is ViT top-1 accuracy, and ‘Original input + SAE’ means clean ViT activations pass through the SAE reconstruction before the next layer. The slight ‘de-attacking’ effect for adversarial inputs suggests that SAEs may filter some adversarial noise, but the model still largely misclassifies, so salient adversarial changes remain available to the network.

Metric	Layer 2	Layer 5	Layer 8	Layer 11
Accuracy				
Original input	1.00	1.00	1.00	1.00
Original input + SAE	0.99	0.99	0.99	0.86
Adv. input	0.00	0.00	0.00	0.00
Adv. input + SAE	0.02	0.01	0.01	0.11
Noisy input	0.98	0.98	0.98	0.98
Noisy input + SAE	0.98	0.98	0.98	0.85
SAE Improvement				
Original input	-0.01	-0.01	-0.01	-0.14
Adv. input	0.02	0.01	0.01	0.11
Noisy input	-0.00	-0.00	-0.00	-0.13

F.3 LAYER-WISE SAE FEATURE DIFFERENCES

This subsection makes the SAE appendix contribution scannable by comparing feature activations across ViT layers for original, ℓ_∞ adversarial, and random noisy inputs. The visualisations are exploratory and identify future directions for studying how attacks alter sparse feature representations. We examine:

- Activation count: Histograms of the number of SAE features above threshold in each layer for original, adversarial, and noisy images, probing whether attacks alter feature sparsity.
- Feature overlap: Counts of shared activated features for original vs. adversarial and original vs. noisy pairs, probing whether attacks activate distinct feature sets.
- Jaccard Similarity: Normalised overlap between activated-feature sets.
- Mean activation: Histograms of mean activation values for above-threshold features, probing whether attacks change feature intensity.
- Distinct Features: Counts of unique features activated across each image set and condition.

F.4 CLS TOKEN SAE FEATURES WITH HIGH ACTIVATION THRESHOLD (0.1)

We first inspect CLS-token SAE features with a high activation threshold of 0.1, focusing on the strongest feature activations. Figures 7 through 10 compare activation counts, feature overlap, Jaccard similarity, and mean activations for original, adversarial, and random noisy inputs across ViT layers.

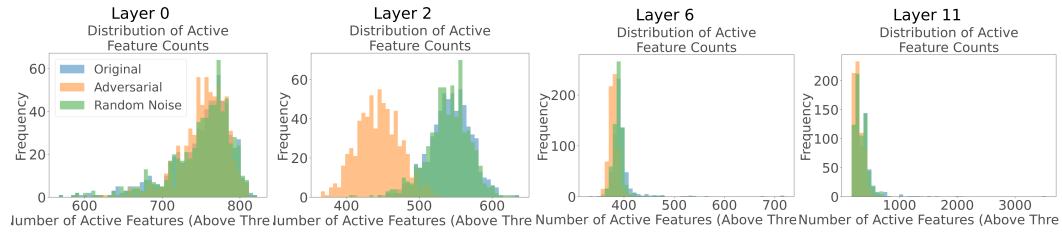


Figure 7: Activation-count histogram for high-threshold CLS-token SAE features (threshold 0.1), comparing original, adversarial, and noisy inputs.

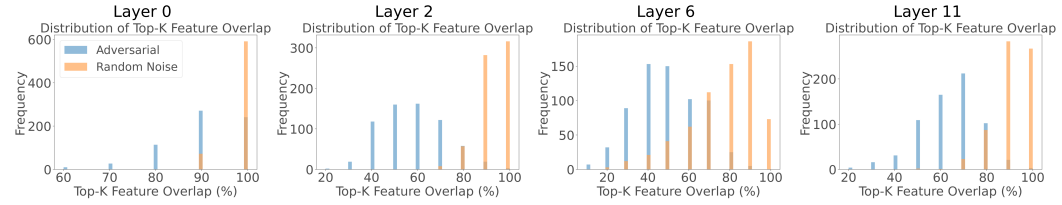


Figure 8: Feature overlap for high-threshold CLS-token SAE features (threshold 0.1), comparing original-attacked and original-noisy image pairs.

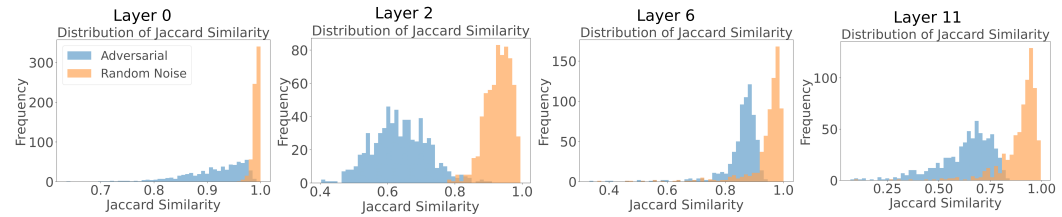


Figure 9: Jaccard similarity for high-threshold CLS-token SAE feature sets (threshold 0.1).

F.5 CLS TOKEN SAE FEATURES WITH LOW ACTIVATION THRESHOLD (0.001)

We then lower the CLS-token threshold to 0.001 to include weaker activations, not only the most intense features. The figures report activation counts (Fig. 11), feature overlap (Fig. 12), Jaccard similarity (Fig. 13), and mean activations (Fig. 14) across layers for original, adversarial, and noisy inputs.

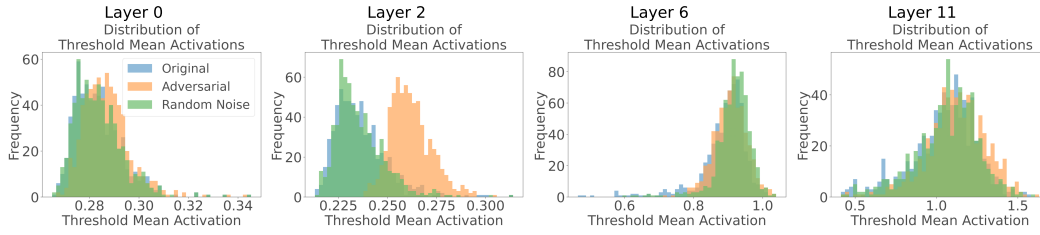


Figure 10: Mean-activation histogram for high-threshold CLS-token SAE features (threshold 0.1).

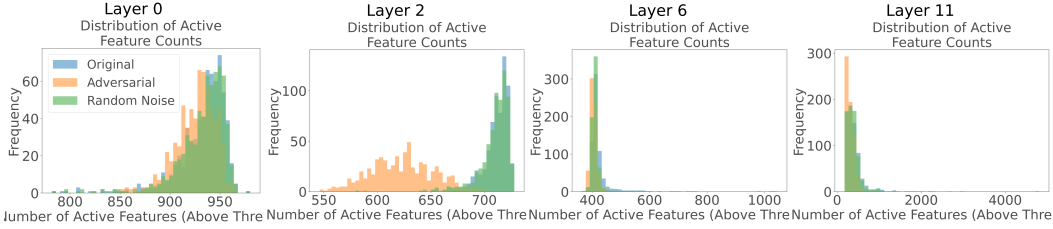


Figure 11: Activation-count histogram for low-threshold CLS-token SAE features (threshold 0.001), including noisy-input comparisons.

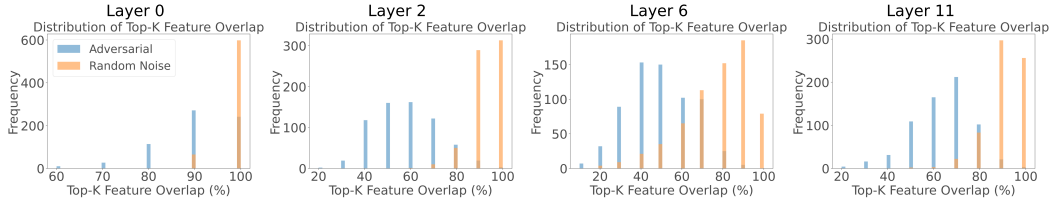


Figure 12: Feature overlap for low-threshold CLS-token SAE features (threshold 0.001), comparing original-attacked and original-noisy image pairs.

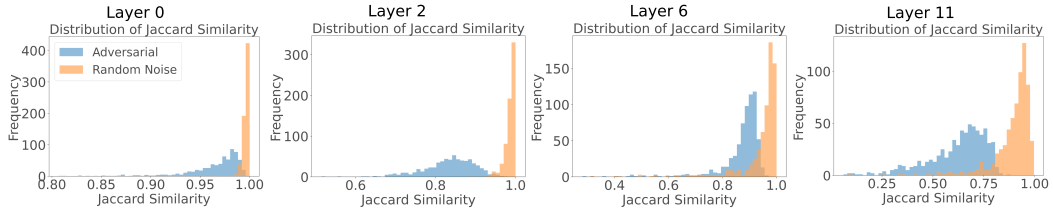


Figure 13: Jaccard similarity for low-threshold CLS-token SAE feature sets (threshold 0.001).

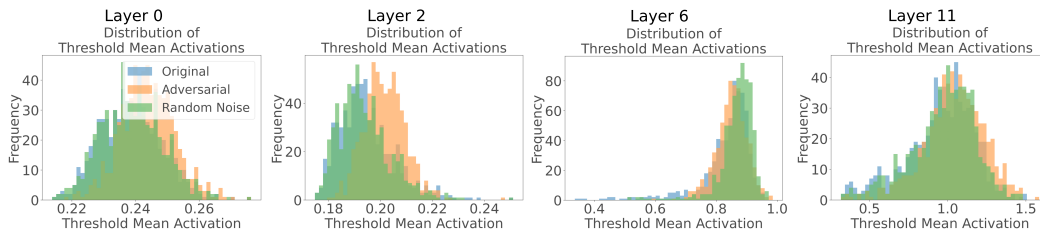


Figure 14: Mean-activation histogram for low-threshold CLS-token SAE features (threshold 0.001), including noisy-input comparisons.

F.6 PATCH TOKEN SAE FEATURES WITH HIGH ACTIVATION THRESHOLD (0.1)

We next inspect SAE features derived from mean patch tokens with the high threshold 0.1. We compare activation counts (Fig. 15), distinct feature counts (Fig. 16), feature overlap (Fig. 17), Jaccard similarity (Fig. 18), and mean activations (Fig. 19) for original, adversarial, and noisy inputs.

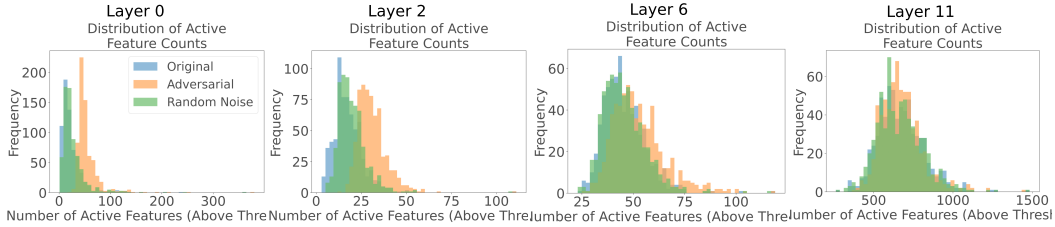


Figure 15: Activation-count histogram for high-threshold patch-token SAE features (threshold 0.1), including noisy-input comparisons.

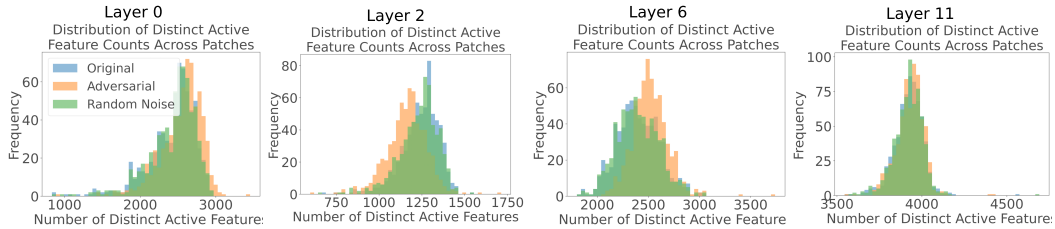


Figure 16: Distinct-feature histogram for high-threshold patch-token SAE features (threshold 0.1), including noisy-input comparisons.

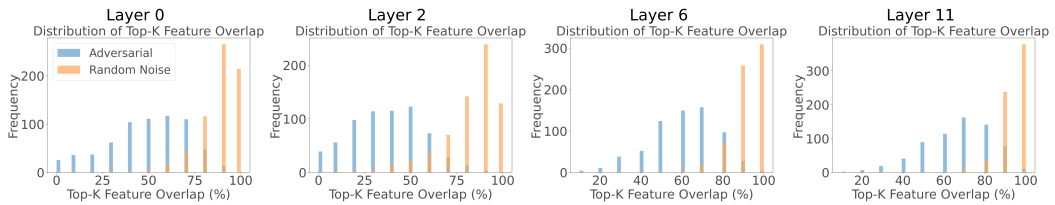


Figure 17: Feature overlap for high-threshold patch-token SAE features (threshold 0.1), comparing original-attacked and original-noisy image pairs.

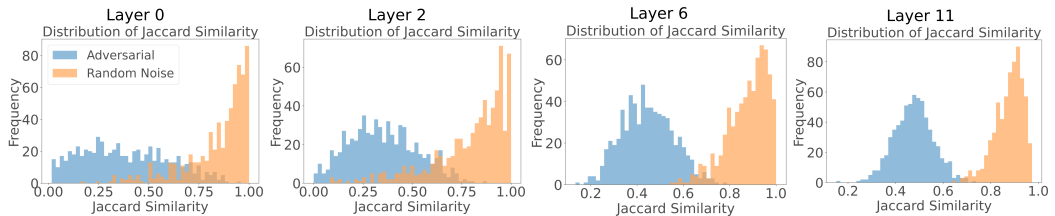


Figure 18: Jaccard similarity for high-threshold patch-token SAE feature sets (threshold 0.1).

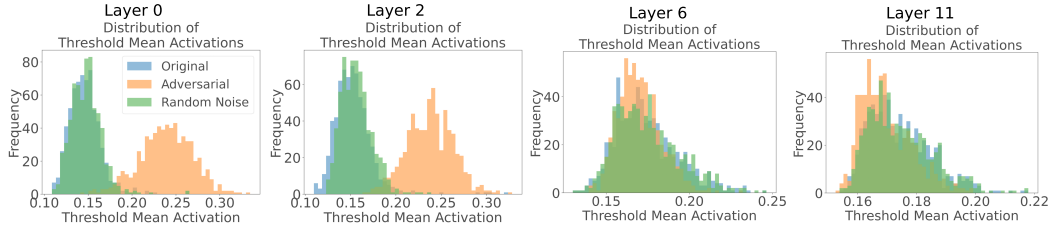


Figure 19: Mean-activation histogram for high-threshold patch-token SAE features (threshold 0.1), including noisy-input comparisons.

F.7 PATCH TOKEN SAE FEATURES WITH LOW ACTIVATION THRESHOLD (0.001)

Finally, we lower the patch-token threshold to 0.001 to include weaker patch-level features. The figures show activation counts (Fig. 20), distinct feature counts (Fig. 21), feature overlap (Fig. 22), Jaccard similarity (Fig. 23), and mean activations (Fig. 24) for original, adversarial, and noisy inputs.

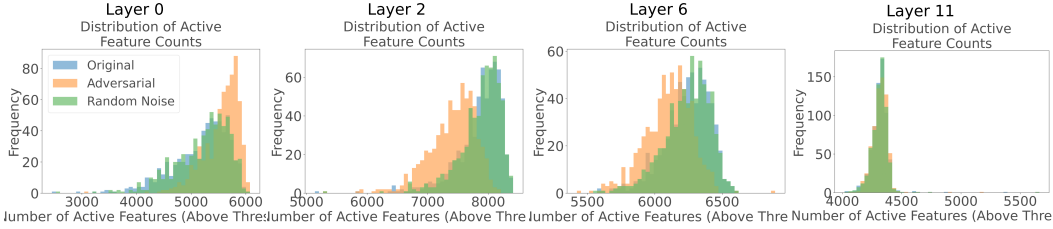


Figure 20: Activation-count histogram for low-threshold patch-token SAE features (threshold 0.001), including noisy-input comparisons.

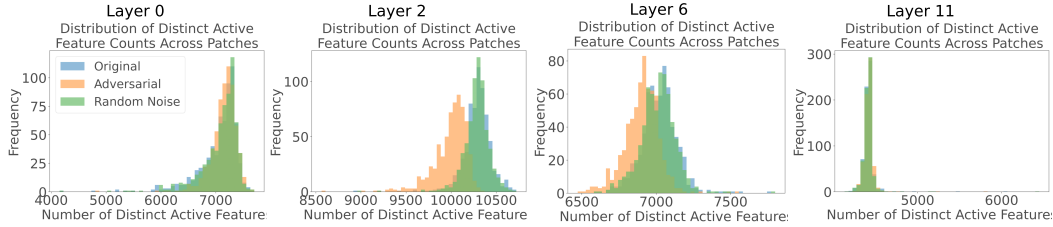


Figure 21: Distinct-feature histogram for low-threshold patch-token SAE features (threshold 0.001), including noisy-input comparisons.

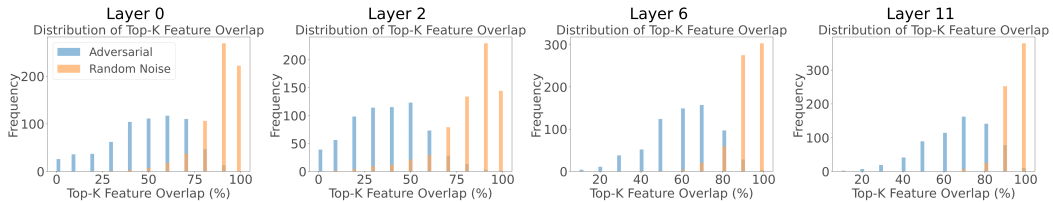


Figure 22: Feature overlap for low-threshold patch-token SAE features (threshold 0.001), comparing original-attacked and original-noisy image pairs.

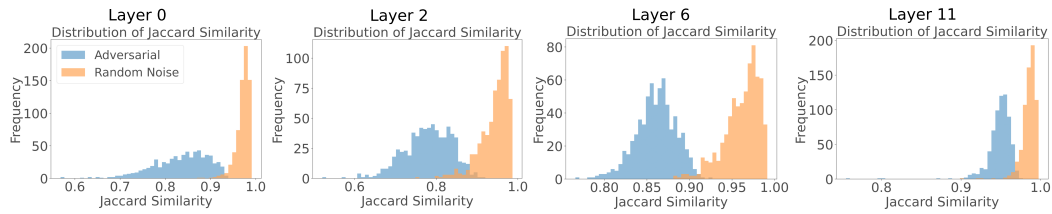


Figure 23: Jaccard similarity for low-threshold patch-token SAE feature sets (threshold 0.001).

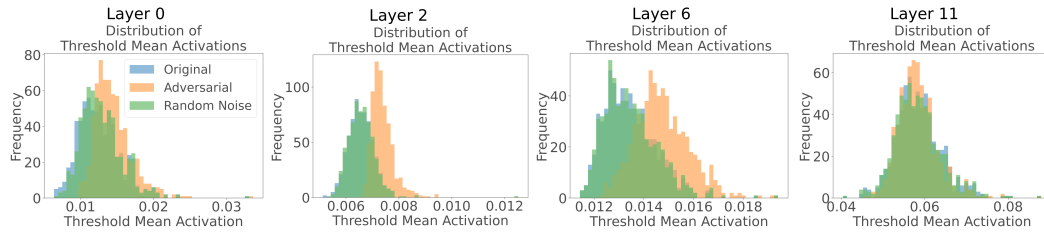


Figure 24: Mean-activation histogram for low-threshold patch-token SAE features (threshold 0.001), including noisy-input comparisons.

G ACRONYMS

AEx adversarial example

CE cross entropy

LRH linear representation hypothesis

MLP Multilayer Perceptron

MSE mean squared error

NN neural network

PGD projected gradient descent

SAE sparse autoencoder

ViT vision transformer